

Tuần 1: Biến số và hàm số

Nội dung bài thực hành:

1. Thao tác tính toán với Command Window.
2. Các hàm cơ bản trong MATLAB.
3. Các phép toán trong MATLAB.

Mọi đóng góp xin liên hệ: tam.doduc@hust.edu.vn

Tóm tắt lý thuyết

Số, biến và hàm số:

Các số trong MATLAB có thể là số dương và âm, số nguyên và phân số, số thực và số phức tạp. Các đặc điểm của biểu diễn số trong MATLAB:

- đơn vị ảo được ký hiệu bởi hai ký tự: i hoặc j ;
- phần nguyên và phần thập phân của số được phân tách bằng dấu chấm;
- những giá trị rất lớn và rất nhỏ được biểu diễn theo cách viết khoa học dùng ký hiệu e .

Định dạng số trong MATLAB:

- *format short* – định dạng ngắn của số (5 chữ số);
- *format short e* – định dạng ngắn của số (5 số với 3 số mũ);
- *format long* – định dạng dài của số (15 chữ số);
- *format long e* – định dạng dài của số (15 số với 3 số mũ).

Biến trong MATLAB:

- Với MATLAB, không cần khai báo biến trước khi sử dụng. Mỗi khi gán một biến mới, MATLAB sẽ định nghĩa dựa vào dữ liệu biến ấy chứa.
- MATLAB hỗ trợ nhiều loại biến, thông thường bạn dùng MATLAB để tính toán với ma trận hoặc vector cả số học hay ký tự.
- Để tạo một biến, chỉ cần gán một giá trị vào cho một tên biến

Quy tắc đặt tên biến:

- ký tự đầu tiên phải là chữ cái
- các ký tự sau có thể là chữ cái hoặc số
- MATLAB phân biệt chữ hoa với chữ thường (var1 khác với Var1)

Một số biến đã được MATLAB định nghĩa sẵn, không được dùng những tên biến này.

- i và j được dùng cho đơn vị phức
- π : số pi, có giá trị 3.1415926...

- ans : lưu kết quả của phép toán vừa thực hiện
- Inf và -Inf là dương và âm vô cực
- NaN : thể hiện Not a Number, không là một số.

Các hàm toán học cơ bản trong MATLAB:

- abs (x): giá trị tuyệt đối của x;
- exp (x): hàm số mũ ;
- log (x), log10 (x), log2 (x): logarit của các số có cơ sở e, 10, 2;
- sqrt (x) : căn bậc hai của x;
- sin (x), cos (x), tan (x), cot (x), sec (x), csc (x) : các hàm lượng giác
- asin (x), acos (x), atan (x), acot (x), asec (x), acsc (x): các hàm lượng giác ngược arcsin x, arcos x, arctg x, arcctg x, arcsec x, arccosec x;
- sinh (x), cosh (x), tanh (x), coth (x), sech (x), csch (x): các hàm hyperbolic sh x, ch x, th x, cth x, sch x, csch x;
- asinh (x), acosh (x), atanh (x), acoth (x), asech (x), acsch (x): hàm ngược của các hàm hyperbolic.

Các phép toán trong MATLAB

Các biểu thức MATLAB sử dụng các toán tử quen thuộc:

- +: Phép cộng;
- -: Phép trừ;
- *: Phép nhân;
- /: Phép chia thông thường;
- \: Phép chia phải;
- ^ : Phép toán lũy thừa;
- ' : Chuyển vị ma trận, chuyển vị liên hợp phức;
- (): xác định ưu tiên của các phép toán.

Các phép toán so sánh

- >: lớn hơn;
- <: nhỏ hơn;
- >=: lớn hơn hoặc bằng;
- <=: nhỏ hơn hoặc bằng;
- ==: so sánh bằng nhau;
- ~=: không bằng nhau.

Các phép toán logic

- &: phép hội (dùng được cho vectơ và ma trận);
- &&: phép hội (đoạn mạch, không dùng cho vectơ hoặc ma trận);

- $|$: phép tuyển (dùng được cho vectơ và ma trận);
- $||$: phép tuyển (đoàn mạch, không dùng cho vectơ hoặc ma trận);
- $\sim$: phép phủ định.

Bài tập

Bài tập 1. Tính bằng tay các biểu thức sau rồi thử lại bằng MATLAB:

- $10/2 \setminus 5 - 3 + 2 * 4$
- $2 + \text{floor}(6/11)/2 - 3$
- $3^{2/4}$
- $2 + \text{ceil}(-6/9) - 3$
- 3^2
- $\text{fix}(-4/9) + \text{fix}(3 * (5/6))$
- $2 + \text{round}(6/9 + 3 * 2)/2 - 3$

% Sinh viên trình bày code lời giải cho bài tập 1 vào đây và sử dụng nút Run để thực thi. Dùng lệnh \help floor trong command window
% để mở hướng dẫn chi tiết cách sử dụng lệnh floor.

Bài tập 2. Cho $a = 36, b = 15$. Tính bằng tay các biểu thức sau rồi thử lại bằng MATLAB:

- $\text{mod}(a, b)$
- $\text{gcd}(a, b)$
- $\text{rem}(a, b)$
- $\text{lcm}(a, b)$

Thử lại với các cặp giá trị (a, b) khác.

% Sinh viên trình bày code lời giải cho bài tập 2 vào đây và sử dụng nút Run để thực thi. Dùng lệnh \help mod trong command window
% để mở hướng dẫn chi tiết cách sử dụng lệnh mod.

Bài tập 3. Dự đoán kết quả những phép toán sau, giải thích và kiểm tra lại bằng MATLAB:

- $1 \& -1$
- $0 < 0.2 < 0.4$
- $13 \& -6$
- $5 > 4 > 3$
- $0 < -20$
- $2 > 3 \& 1$

% Sinh viên trình bày code lời giải cho bài tập 3 vào đây và sử dụng nút Run để thực thi.

Tuần 2: Véc tơ

Nội dung bài thực hành:

1. Véc tơ
2. Đa thức

Sinh viên lưu file dưới theo định dạng "Tên không dấu + MSSV.mlx" và nộp bài trên nhóm teams của lớp ở mục assignments 2.

Mọi đóng góp xin liên hệ: tam.doduc@hust.edu.vn

Tóm tắt lý thuyết

Véc tơ

Khởi tạo véc tơ:

- Để khởi tạo véc tơ hàng chứa các giá trị rời rạc, các phần tử của véc tơ phải nằm trong cặp ngoặc vuông "[" "]" và được ngăn cách với nhau bởi dấu phẩy ",", hoặc khoảng trắng.
- Để khởi tạo một véc tơ hàng chứa các giá trị liên tiếp hoặc cách nhau một giá trị nhất định (bước nhảy), MATLAB sử dụng toán tử ":", khi đó giá trị đầu và cuối của véc tơ không nhất thiết phải đặt trong dấu ngoặc vuông.
- Hàm **linspace** tạo ra một véc tơ với khoảng cách tuyến tính. Nó tương tự như toán tử hai chấm ":", nhưng xác định trước số điểm chia n.
- Hàm **logspace** tạo ra một véc tơ với khoảng cách logarit, cụ thể $y = \text{logspace}(a, b)$ tạo ra một véc tơ hàng y gồm 50 điểm trong khoảng 10^a và 10^b .

Chỉ số véc tơ:

Giá trị của một phần tử tại một vị trí bất kỳ trong véc tơ được truy xuất thông qua chỉ số. Trong MATLAB, chỉ số luôn bắt đầu từ 1 và có thể là một giá trị đơn hoặc một mảng.

- $X(k)$: lấy ra phần tử thứ k của véc tơ X.
- $X(\text{danh sách các vị trí})$: lấy ra các phần tử ở các vị trí trong danh sách.
- $X[k]=[]$: xóa phần tử thứ k của véc tơ X.

Các thành phần của véc tơ có thể được truy xuất thông qua các biểu thức logic.

tên véc tơ (biểu thức logic)

Biểu thức logic thường được sử dụng với các hàm any, all và find.

- any: Kiểm tra xem có tồn tại một phần tử của véc tơ thỏa mãn điều kiện nào đó không. Kết quả trả về 1 nếu đúng và 0 nếu sai.
- all: Kiểm tra xem tất cả các phần tử của véc tơ thỏa mãn điều kiện nào đó không. Kết quả trả về 1 nếu đúng và 0 nếu sai.

- find: Trả về chỉ số của véc tơ thỏa mãn một điều kiện nào đó.

Các phép toán cơ bản trên véc tơ.

Giả sử a, b là hai véc tơ có cùng kích thước. Ta có các phép toán cơ bản sau:

- $a + b, a - b$: Cộng, trừ 2 véc tơ.
- $\text{dot}(a, b)$: Nhân vô hướng 2 véc tơ.
- $a * b$: Nhân từng phần tử tương ứng của 2 véc tơ cho nhau và trả về véc tơ kết quả cùng chiều.
- $a.^b$: Trả về véc tơ dạng $(a_1^{b_1}, a_2^{b_2}, \dots, a_n^{b_n})$.
- $a.^n$: Lũy thừa từng phần tử của véc tơ số mũ n .
- $\text{sort}(a); \text{sort}(a, 'decend')$: Sắp xếp mảng theo thứ tự tăng, giảm dần.
- $a ./ b$: Chia phải, nghĩa là, chia từng phần tử tương ứng của a cho b .
- $a \setminus b$: Chia trái, nghĩa là, chia từng phần tử tương ứng của b cho a .
- $a \& b$: Phép hợp giữa 2 véc tơ, trả về véc tơ logic cùng chiều.
- $a | b$: Phép tuyển giữa 2 véc tơ, trả về véc tơ logic cùng chiều.
- $\sim a$: Phủ định của a .
- $\text{intersect}(a, b)$: Phép giao hai tập hợp a, b .
- $\text{union}(a, b)$: Phép hợp hai tập hợp a, b .
- $\text{setdiff}(a, b)$: Hiệu hai tập hợp a, b , lấy ra cả phần tử thuộc a và không thuộc b .

Đa thức

Biểu diễn của đa thức trong Matlab.

Một đa thức bao giờ cũng có dạng

$$f(x) = a_n x^n + a_{n-1} x^{n-1} + \dots + a_1 x + a_0; (a_n \neq 0).$$

Trong Matlab một đa thức được biểu diễn bởi một véc tơ hàng có các thành phần là các hệ số theo thứ tự bậc giảm dần, kể cả hệ số 0.

Một số phép toán cơ bản về đa thức trong Matlab

- $\text{polyval}(p, x)$: tính giá trị của đa thức p tại điểm x .
- $r = \text{roots}(p)$: tìm nghiệm của đa thức p trả về cho véc tơ cột r .
- $p = \text{poly}(r)$: tìm đa thức khi biết trước nghiệm.
- $p + q$: cộng hai đa thức p và q .
- $c = \text{conv}(a, b)$: nhân hai đa thức a và b .
- $[q, r] = \text{deconv}(u, v)$: chia đa thức u cho v , kết quả trả về q là đa thức thương và r là đa thức dư.
- $k = \text{polyder}(p)$: đạo hàm của đa thức p , kết quả trả về cho k .
- $g = \text{polyint}(h, k)$: tính nguyên hàm của đa thức h , trong đó k là hằng số tích phân, mặc định là 0.

Bài tập 1. Nhập vào vector $x = [3 \ 1 \ 5 \ 7 \ 9 \ 2 \ 6]$, dự đoán kết quả của các dòng lệnh sau và thử lại bằng MATLAB:

1. $x(3)$
2. $x(1 : 7)$
3. $x(1 : \text{end})$
6. $x([1 \ 6 \ 2 \ 1 \ 1])$
4. $x(1 : \text{end} - 1)$
7. $\text{sum}(x), \text{min}(x), \text{max}(x).$
5. $x(6 : -2 : 1)$

% Sinh viên trình bày code lời giải cho bài tập 1 vào đây và sử dụng nút Run Section để thực thi.

Bài tập 2. Cho $x = [1 \ 5 \ 2 \ 8 \ 9 \ 0 \ 1]$ và $y = [5 \ 2 \ 2 \ 6 \ 0 \ 0 \ 2]$ giải thích kết quả của các dòng lệnh sau:

1. $x > y$
2. $y \geq x$
3. $(x > y)!(y < x)$
4. $y < x$
5. $x \mid y$
6. $(x > y) \& (y < x)$
7. $x == y$
8. $x \& y$
9. $x \leq y$
10. $x \& -y$

% Sinh viên trình bày code lời giải cho bài tập 2 vào đây và sử dụng nút Run Section để thực thi.
% Lời giải thích ghi dưới dạng comment sử dụng dấu %

Bài tập 3. Cho hai vector $a = [1 \ 0 \ 2]$ và $a = [0 \ 2 \ 2]$ xác định giá trị của các biểu thức sau, giải thích và sau đó kiểm tra lại bằng MATLAB:

1. $a = b$
5. $a \mid (a)$
2. $a < b$
6. $b \& (b)$
3. $a < b < a$
7. $a = b == a$
4. $a < b < b$

% Sinh viên trình bày code lời giải cho bài tập 3 vào đây và sử dụng nút Run Section để thực thi.
% Lời giải thích ghi dưới dạng comment sử dụng dấu %

Bài tập 4. Cho $x = 1 : 10$ và $y = [3 \ 1 \ 5 \ 6 \ 8 \ 2 \ 9 \ 4 \ 7 \ 0]$, dự đoán kết quả các dòng lệnh, giải thích và thử lại bằng MATLAB:

1. $(x > 3) \& (x < 8)$
2. $x(x > 5)$
3. $y(x \leq 4)$
4. $x((x < 2) \mid (x \geq 8))$
5. $y((x < 2) \mid (x \geq 8))$
6. $x(y < 0)$

% Sinh viên trình bày code lời giải cho bài tập 4 vào đây và sử dụng nút Run Section để thực thi.
% Lời giải thích ghi dưới dạng comment sử dụng dấu %

Bài tập 5. Cho $A = [2 \ 7 \ 9 \ 7; \ 3 \ 1 \ 5 \ 6; \ 8 \ 1 \ 2 \ 5]$, $x = [1 \ 4 \ 8]$, và $y = [2 \ 1 \ 5]$. Xét xem dòng lệnh nào hợp lệ, dự đoán kết quả, giải thích và thử lại bằng MATLAB:

1. $[x; y]$
2. $A(:)$
3. $[x; y]$
4. $[A; A(\text{end}, :)]$
5. $A(:, [1, 4])$
6. $A(1 : 3, :)$
7. $A([2 \ 3], [3 \ 1])$
8. $[A; A(1 : 2, :)]$

% Sinh viên trình bày code lời giải cho bài tập 5 vào đây và sử dụng nút Run Section để thực thi.
% Lời giải thích ghi dưới dạng comment sử dụng dấu %

Bài tập 6. Cho các vector cột $x = [3 \ 2 \ 6 \ 8]'$, $y = [4 \ 1 \ 3 \ 5]'$.

1. Lấy tổng các phần tử của x thêm vào từng phần tử của y .
2. Lũy thừa mỗi phần tử của x với số mũ là các phần tử của y tương ứng.
3. Chia các phần tử tương ứng của y cho x .
4. Nhân các phần tử tương ứng của x và y và gán kết quả cho vector w .
5. Tính $x.*y - w$.

6. Tính tích vô hướng của x và y .

% Sinh viên trình bày code lời giải cho bài tập 6 vào đây và sử dụng nút Run Section để thực thi.

Bài tập 7. Tạo vector $x = \text{randperm}(35)$ (tạo ngẫu nhiên một hoán vị từ 1 đến 35). Viết các lệnh để tính hàm sau sử dụng chỉ số logic:

$$y(x) = \begin{cases} 2, & \text{khi } x < 6, \\ x - 4, & \text{khi } 6 \leq x \leq 20, \\ 36 - x, & \text{khi } 20 \leq x \leq 35. \end{cases}$$

% Sinh viên trình bày code lời giải cho bài tập 7 vào đây và sử dụng nút Run Section để thực thi.

Bài tập 8. Cho vectơ $x = [3, 15, 9, 12, -1, 0, -12, 9, 6, 1]$.

1. Viết lệnh chuyển các giá trị dương thành 0 .
2. Chuyển các bội số của 3 thành số 3 .
3. Nhân các giá trị chẵn với 5 .
4. Chuyển các giá trị nhỏ hơn trung bình cộng thành 0 .

% Sinh viên trình bày code lời giải cho bài tập 8 vào đây và sử dụng nút Run Section để thực thi.

Bài tập 9. Cho $x = [2 \ -5 \ 1 \ 6]$. Viết các lệnh thực hiện:

1. Cộng thêm 16 vào tất cả các phần tử.
2. Cộng thêm 3 vào tất cả các phần tử ở vị trí lẻ.
3. Lấy căn bậc 2 của tất cả các phần tử có giá trị dương.
4. Bình phương tất cả các phần tử.

% Sinh viên trình bày code lời giải cho bài tập 9 vào đây và sử dụng nút Run Section để thực thi.

Bài tập 10. Cho đa thức $P(x) = 10x^5 + 3x^3 - 2x^2 + 4x + 9$.

1. Khai báo đa thức đã cho sử dụng vectơ P .
2. Tính giá trị của đa thức đã cho tại $x = 2$.
3. Tìm nguyên hàm của đa thức.

4. Tính đạo hàm của đa thức.
5. Tìm nghiệm của đa thức.

% Sinh viên trình bày code lời giải cho bài tập 10 vào đây và sử dụng nút Run Section để thực thi.

Bài tập 11. Cho 2 đa thức $p(x) = 3x^3 - x^2 + x$ và $q(x) = 2x^2 - 1$.

1. Khai báo 2 đa thức p và q .
2. Tính tổng $p + q$.
3. Tính tích $p(x)q(x)$.
4. Tìm thương và dư trong phép chia $p(x)$ cho $q(x)$.

% Sinh viên trình bày code lời giải cho bài tập 11 vào đây và sử dụng nút Run Section để thực thi.

Tuần 3: Ma trận

Nội dung bài thực hành:

1. Ma trận
2. Vẽ đồ thị

Sinh viên lưu file dưới theo định dạng "Tên không dấu + MSSV.mlx" và nộp bài trên nhóm teams của lớp ở mục assignments 3.

Mọi đóng góp xin liên hệ: tam.doduc@hust.edu.vn

Tóm tắt lý thuyết

Khai báo:

Khi nhập ma trận trong môi trường dòng lệnh (command window hoặc editor) ta phải tuân theo các quy luật sau:

- Ngăn cách các phần tử trên 1 hàng bằng khoảng trắng hoặc dấu phẩy ", ".
- Dùng dấu ";" để kết thúc 1 hàng.
- Bao các phần tử của ma trận bởi cặp dấu [].

```
A = [1 3 4; 5 6 7; 9 8 1]
```

```
A = 3x3
    1     3     4
    5     6     7
    9     8     1
```

Tạo ma trận sử dụng lệnh MATLAB

- `zeros(m,n)`: tạo ma trận m dòng n cột tất cả phần tử bằng 0.
- `ones(m,n)`: tạo ma trận m dòng n cột tất cả phần tử bằng 1.
- `rand(m,n)`: tạo ma trận m dòng n cột tất cả phần tử có phân bố đều trên $[0, 1]$.
- `randn(m,n)`: tạo ma trận m dòng n cột với các phần tử ma trận có phân bố chuẩn.
- `randi([a,b],m,n)`: tạo ma trận m dòng n cột với phần tử bằng có giá trị nguyên phân phối đều từ đoạn $[a, b]$.
- `magic(n)`: tạo ra ma trận cấp n gồm các số nguyên tử 1 đến n^2 với tổng các hàng bằng tổng các cột.
- `pascal(n)`: tạo ra ma trận có các phần tử lấy từ tam giác Pascal.
- `eye(n)`: tạo ra ma trận đơn vị cấp n .

```
zeros(3,4)
```

```
ans = 3x4
    0     0     0     0
    0     0     0     0
    0     0     0     0
```

```
ones(3,4)
```

```
ans = 3x4
     1     1     1     1
     1     1     1     1
     1     1     1     1
```

```
rand(3,4)
```

```
ans = 3x4
     0.1058     0.4046     0.7635     0.9329
     0.1097     0.4484     0.6279     0.9727
     0.0636     0.3658     0.7720     0.1920
```

```
randn(3,4)
```

```
ans = 3x4
    -2.3290     0.0668    -0.0692     0.2458
     0.9019     0.0355    -0.5073     0.0700
    -1.8356     2.2272     0.2358    -0.6086
```

```
randi([-10,5],3,4)
```

```
ans = 3x4
    -8    -10     -3     1
    -1     2      1    -4
    -6     -7     -5     0
```

```
magic(3)
```

```
ans = 3x3
     8     1     6
     3     5     7
     4     9     2
```

```
pascal(3)
```

```
ans = 3x3
     1     1     1
     1     2     3
     1     3     6
```

```
eye(3)
```

```
ans = 3x3
     1     0     0
     0     1     0
     0     0     1
```

Truy xuất phần tử của ma trận

Quy tắc truy cập đến các phần tử của ma trận thông qua chỉ số:

- Phần tử ở hàng i , cột j của ma trận A là $A(i, j)$.
- Trong MATLAB, chỉ số cuối cùng của hàng hay cột của ma trận hoặc véc tơ có thể thay thế bởi `end`.
- Ta cũng có thể tham chiếu tới phần tử mảng nhờ một chỉ số. Phần tử $A(i, j)$ ứng với $A(k)$ với $k = i + (j - 1)m$. Để chuyển từ chỉ số ma trận sang chỉ số mảng một chiều dùng lệnh:

$$k = \text{sub2ind}(\text{size}(A), i, j).$$

- Ngược lại, để chuyển từ chỉ số mảng 1 chiều sang chỉ số ma trận dùng lệnh:

$$[i, j] = \text{ind2sub}(\text{size}(A), k).$$

- Lệnh $A(1:k, j)$ tham chiếu đến k phần tử đầu của cột j của ma trận.
- Lệnh $A(:, j)$ và $A(i, :)$ tham chiếu tương ứng đến cột j và dòng i của ma trận.
- Để xác định kích thước của ma trận ta có thể dùng lệnh `length` (trả về kích thước lớn nhất) hay lệnh `size` (trả về số hàng và số cột).

Phần tử của ma trận có thể truy xuất thông qua biểu thức logic

tên ma trận(biểu thức logic).

```
A=randi([-10,20],4,5)
```

```
A = 4x5
```

```
11      3      3      2     -4
 3     -2     17     15     14
-10     -4      2     13     19
 0     15     13      1      0
```

```
A(A<0)
```

```
ans = 4x1
```

```
-10
 -2
 -4
 -4
```

```
A((A>0)&(A<10))
```

```
ans = 6x1
```

```
3
 3
 3
 2
 2
 1
```

Các lệnh thao tác với ma trận

Lệnh `diag` dùng để tạo ra ma trận đường chéo và rút ra đường chéo của ma trận.

- `diag(v, k)`: Nếu v là véc tơ có độ dài n thì lệnh này tạo ra một ma trận vuông kích thước $n + |k|$ với các phần tử của v nằm trên đường chéo thứ k .
- `diag(X, k)`: Nếu X là một ma trận thì kết quả trả về véc tơ cột mà các phần tử là đường chéo k của ma trận X .
- `diag(X)`: Nếu X là một ma trận thì kết quả trả về véc tơ cột mà các phần tử là đường chéo chính của ma trận X .

- `diag(diag(X))`: Nếu X là một ma trận thì kết quả trả về là một ma trận đường chéo, các phần tử là đường chéo chính của X .

```
v=[1 2 3]
```

```
v = 1x3
     1     2     3
```

```
diag(v)
```

```
ans = 3x3
     1     0     0
     0     2     0
     0     0     3
```

```
diag(v,2)
```

```
ans = 5x5
     0     0     1     0     0
     0     0     0     2     0
     0     0     0     0     3
     0     0     0     0     0
     0     0     0     0     0
```

```
diag(v,-1)
```

```
ans = 4x4
     0     0     0     0
     1     0     0     0
     0     2     0     0
     0     0     3     0
```

```
A = randi(5,6)
```

```
A = 6x6
     4     5     3     1     3     4
     3     3     4     1     1     3
     5     5     5     1     1     5
     4     3     4     4     5     5
     1     1     2     3     3     5
     5     1     3     1     5     1
```

```
diag(A)
```

```
ans = 6x1
     4
     3
     5
     4
     3
     1
```

MATLAB cung cấp các phép toán đại số tuyến tính với ma trận

$\text{Det}(A)$: Tính định thức của ma trận vuông A .

$A + B$: Tổng của hai ma trận A và B .

$A - B$: Hiệu của hai ma trận A và B .

$A * B$: Tích của hai ma trận A và B .

$A. * B$: Nhân từng phần tử tương ứng của hai ma trận.

$X = A/B$: Chia hai ma trận ($X * B = A$).

$\text{inv}(A)$: Nghịch đảo của ma trận A .

$A.^n$: Lũy thừa bậc n từng phần tử của A .

Nếu đem ma trận A cộng (trừ) với một số vô hướng a thì số a được cộng (trừ) vào từng phần tử của A .

A

A = 6×6

4	5	3	1	3	4
3	3	4	1	1	3
5	5	5	1	1	5
4	3	4	4	5	5
1	1	2	3	3	5
5	1	3	1	5	1

A+3

ans = 6×6

7	8	6	4	6	7
6	6	7	4	4	6
8	8	8	4	4	8
7	6	7	7	8	8
4	4	5	6	6	8
8	4	6	4	8	4

A-2

ans = 6×6

2	3	1	-1	1	2
1	1	2	-1	-1	1
3	3	3	-1	-1	3
2	1	2	2	3	3
-1	-1	0	1	1	3
3	-1	1	-1	3	-1

Nếu đem ma trận A cộng (trừ) với một vectơ hàng b có số phần tử bằng số cột của A thì vectơ b được cộng (trừ) vào từng dòng của A .

A

A = 6×6

4	5	3	1	3	4
3	3	4	1	1	3
5	5	5	1	1	5
4	3	4	4	5	5
1	1	2	3	3	5
5	1	3	1	5	1

```
b = [1 2 3 4 5 6]
```

```
b = 1×6  
    1     2     3     4     5     6
```

```
A+b
```

```
ans = 6×6  
    5     7     6     5     8    10  
    4     5     7     5     6     9  
    6     7     8     5     6    11  
    5     5     7     8    10    11  
    2     3     5     7     8    11  
    6     3     6     5    10     7
```

Nếu đem ma trận A cộng (trừ) với một vectơ cột c có số phần tử bằng số dòng của A thì vectơ c được cộng (trừ) vào từng cột của A .

```
A
```

```
A = 6×6  
    4     5     3     1     3     4  
    3     3     4     1     1     3  
    5     5     5     1     1     5  
    4     3     4     4     5     5  
    1     1     2     3     3     5  
    5     1     3     1     5     1
```

```
c = [1 2 3 4 5 6]'
```

```
c = 6×1  
    1  
    2  
    3  
    4  
    5  
    6
```

```
A+c
```

```
ans = 6×6  
    5     6     4     2     4     5  
    5     5     6     3     3     5  
    8     8     8     4     4     8  
    8     7     8     8     9     9  
    6     6     7     8     8    10  
   11     7     9     7    11     7
```

Vẽ đồ thị với MATLAB

Lệnh plot vẽ đồ thị 2D trong MATLAB:

```
>> plot(vectơ chứa hoành độ, vectơ chứa tung độ).
```

```
% vẽ đồ thị hàm số y = sin(x) trên khoảng [-pi,pi]  
x = linspace(-pi,pi)
```

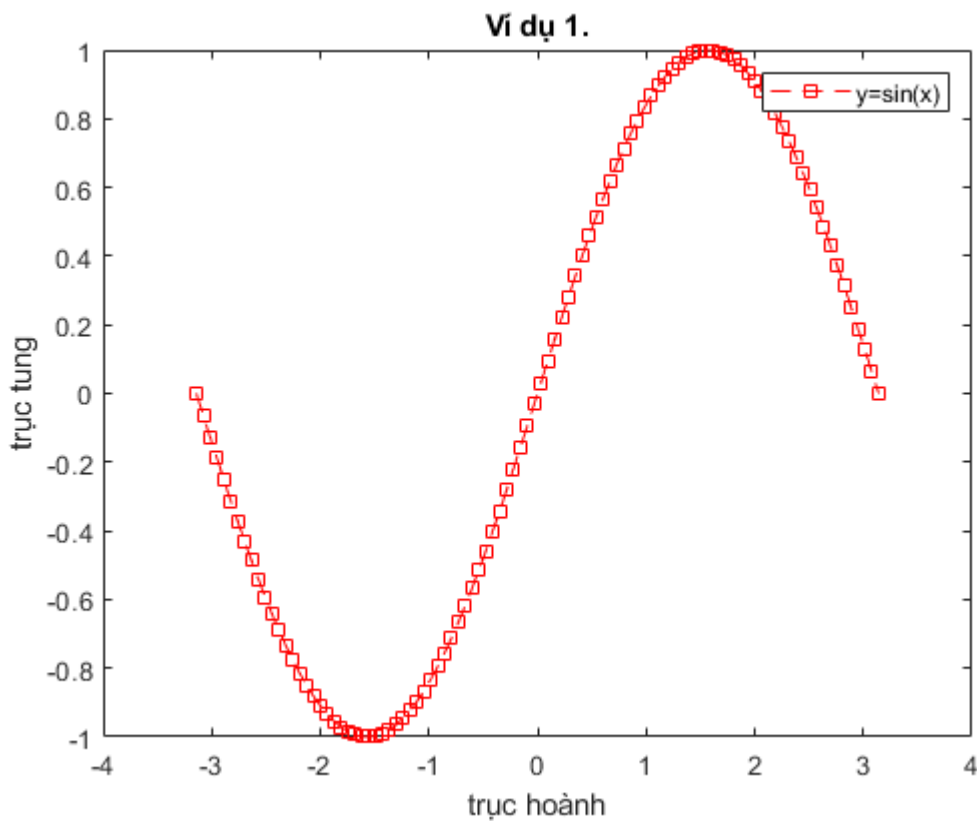
```
x = 1×100  
   -3.1416   -3.0781   -3.0147   -2.9512   -2.8877   -2.8243   -2.7608   -2.6973 ...
```



```
y = sin(x)
```

```
y = 1×100  
-0.0000 -0.0634 -0.1266 -0.1893 -0.2511 -0.3120 -0.3717 -0.4298 ...
```

```
plot(x,y,'--rs')  
xlabel('trục hoành')  
ylabel('trục tung')  
title('Ví dụ 1.')  
legend('y=sin(x)')
```



MATLAB cung cấp các lệnh để vẽ đồ thị 3D

`plot3(x, y, z)` hoặc `surf(x, y, z)`;

trong đó x, y, z là các ma trận chứa hoành độ, tung độ và cao độ của các điểm nằm trên đồ thị.

Vẽ đồ thị mặt cong trong MATLAB] Vẽ đồ thị hàm số $z = xe^{-x^2-y^2}$ trong khoảng $[-2, 2] \times [-2, 2]$..

```
[X,Y] = meshgrid(linspace(-2,2,200))
```

```
X = 200×200  
-2.0000 -1.9799 -1.9598 -1.9397 -1.9196 -1.8995 -1.8794 -1.8593 ...  
-2.0000 -1.9799 -1.9598 -1.9397 -1.9196 -1.8995 -1.8794 -1.8593  
-2.0000 -1.9799 -1.9598 -1.9397 -1.9196 -1.8995 -1.8794 -1.8593  
-2.0000 -1.9799 -1.9598 -1.9397 -1.9196 -1.8995 -1.8794 -1.8593  
-2.0000 -1.9799 -1.9598 -1.9397 -1.9196 -1.8995 -1.8794 -1.8593
```

```

-2.0000 -1.9799 -1.9598 -1.9397 -1.9196 -1.8995 -1.8794 -1.8593
-2.0000 -1.9799 -1.9598 -1.9397 -1.9196 -1.8995 -1.8794 -1.8593
-2.0000 -1.9799 -1.9598 -1.9397 -1.9196 -1.8995 -1.8794 -1.8593
-2.0000 -1.9799 -1.9598 -1.9397 -1.9196 -1.8995 -1.8794 -1.8593
-2.0000 -1.9799 -1.9598 -1.9397 -1.9196 -1.8995 -1.8794 -1.8593
:
:
Y = 200x200
-2.0000 -2.0000 -2.0000 -2.0000 -2.0000 -2.0000 -2.0000 -2.0000 ...
-1.9799 -1.9799 -1.9799 -1.9799 -1.9799 -1.9799 -1.9799 -1.9799
-1.9598 -1.9598 -1.9598 -1.9598 -1.9598 -1.9598 -1.9598 -1.9598
-1.9397 -1.9397 -1.9397 -1.9397 -1.9397 -1.9397 -1.9397 -1.9397
-1.9196 -1.9196 -1.9196 -1.9196 -1.9196 -1.9196 -1.9196 -1.9196
-1.8995 -1.8995 -1.8995 -1.8995 -1.8995 -1.8995 -1.8995 -1.8995
-1.8794 -1.8794 -1.8794 -1.8794 -1.8794 -1.8794 -1.8794 -1.8794
-1.8593 -1.8593 -1.8593 -1.8593 -1.8593 -1.8593 -1.8593 -1.8593
-1.8392 -1.8392 -1.8392 -1.8392 -1.8392 -1.8392 -1.8392 -1.8392
-1.8191 -1.8191 -1.8191 -1.8191 -1.8191 -1.8191 -1.8191 -1.8191
:
:

```

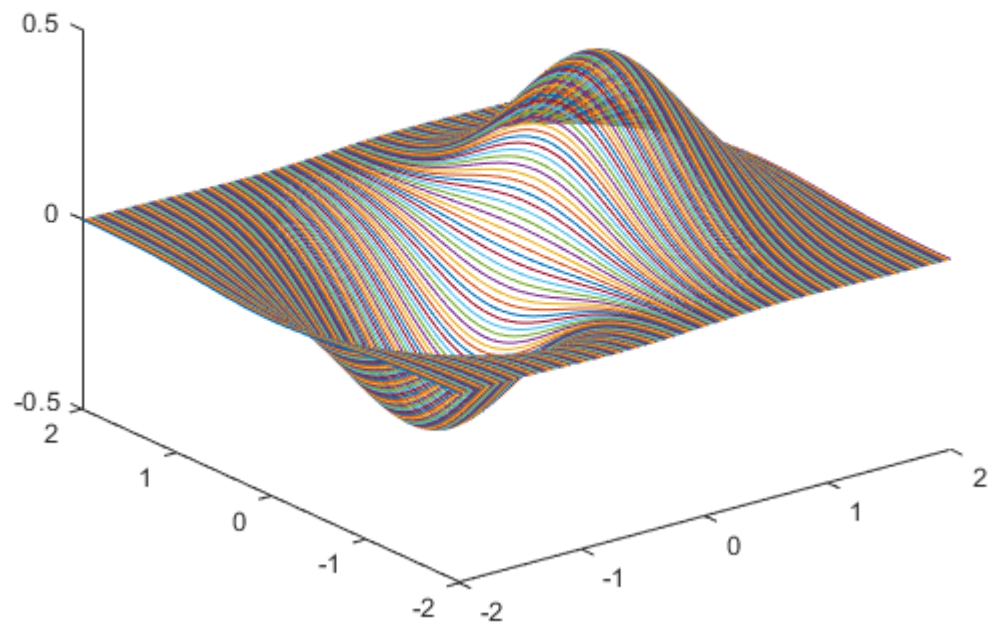
```
Z = X.*exp(-X.^2-Y.^2)
```

```

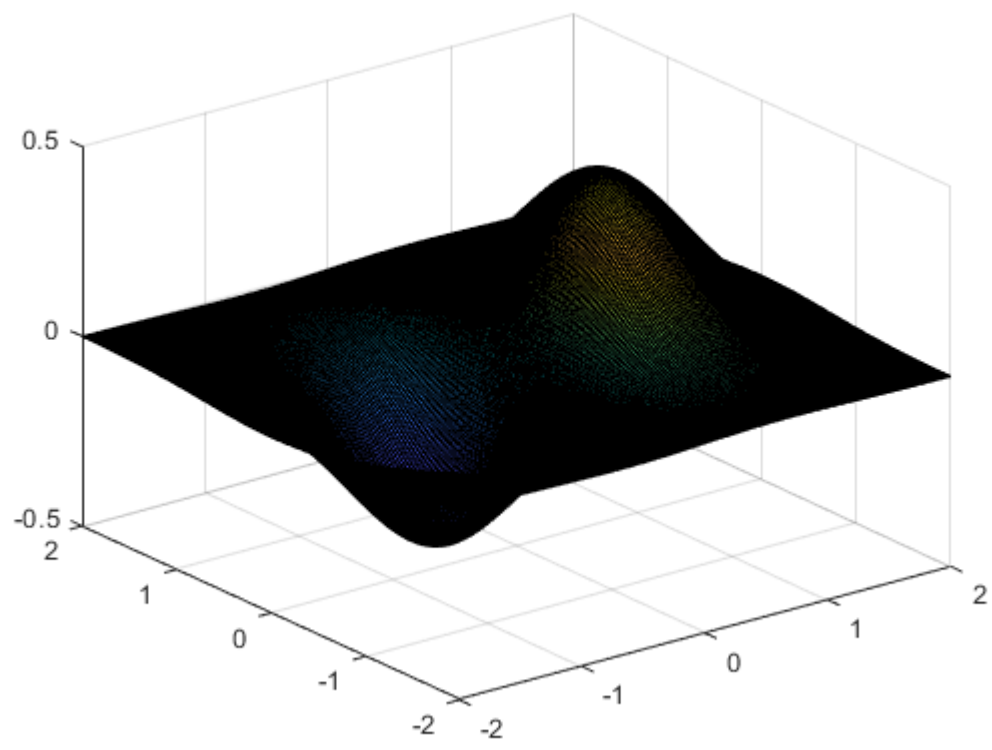
Z = 200x200
-0.0007 -0.0007 -0.0008 -0.0008 -0.0009 -0.0009 -0.0010 -0.0011 ...
-0.0007 -0.0008 -0.0008 -0.0009 -0.0010 -0.0010 -0.0011 -0.0012
-0.0008 -0.0008 -0.0009 -0.0010 -0.0010 -0.0011 -0.0012 -0.0013
-0.0009 -0.0009 -0.0010 -0.0010 -0.0011 -0.0012 -0.0013 -0.0014
-0.0009 -0.0010 -0.0011 -0.0011 -0.0012 -0.0013 -0.0014 -0.0015
-0.0010 -0.0011 -0.0011 -0.0012 -0.0013 -0.0014 -0.0015 -0.0016
-0.0011 -0.0011 -0.0012 -0.0013 -0.0014 -0.0015 -0.0016 -0.0017
-0.0012 -0.0012 -0.0013 -0.0014 -0.0015 -0.0016 -0.0017 -0.0018
-0.0012 -0.0013 -0.0014 -0.0015 -0.0016 -0.0017 -0.0019 -0.0020
-0.0013 -0.0014 -0.0015 -0.0016 -0.0018 -0.0019 -0.0020 -0.0021
:
:

```

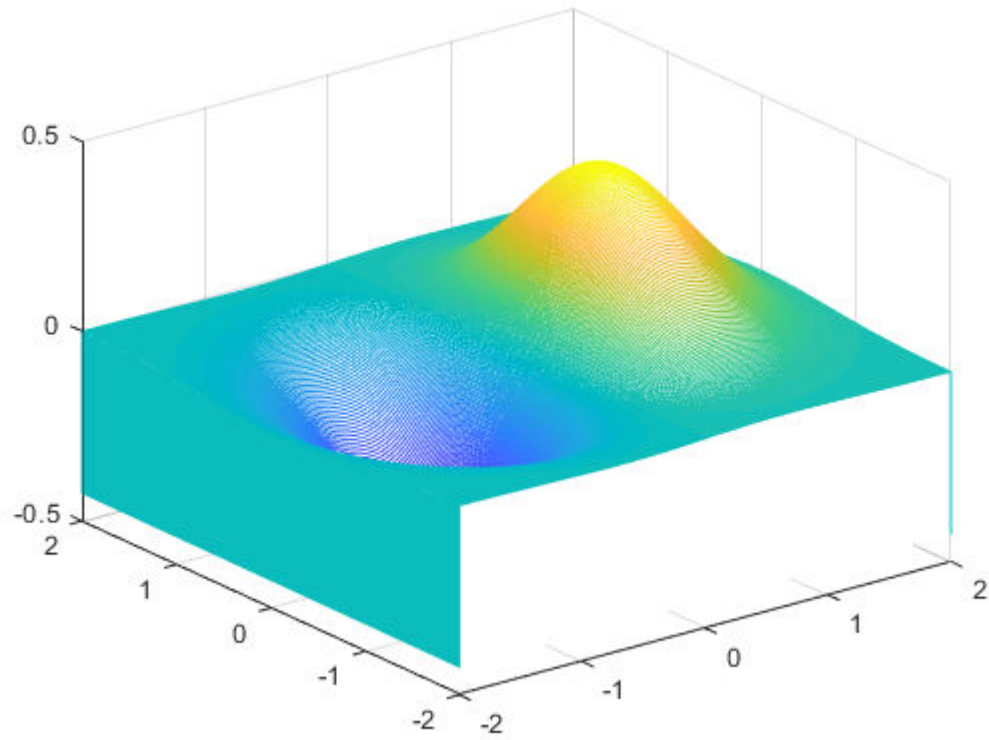
```
plot3(X,Y,Z)
```



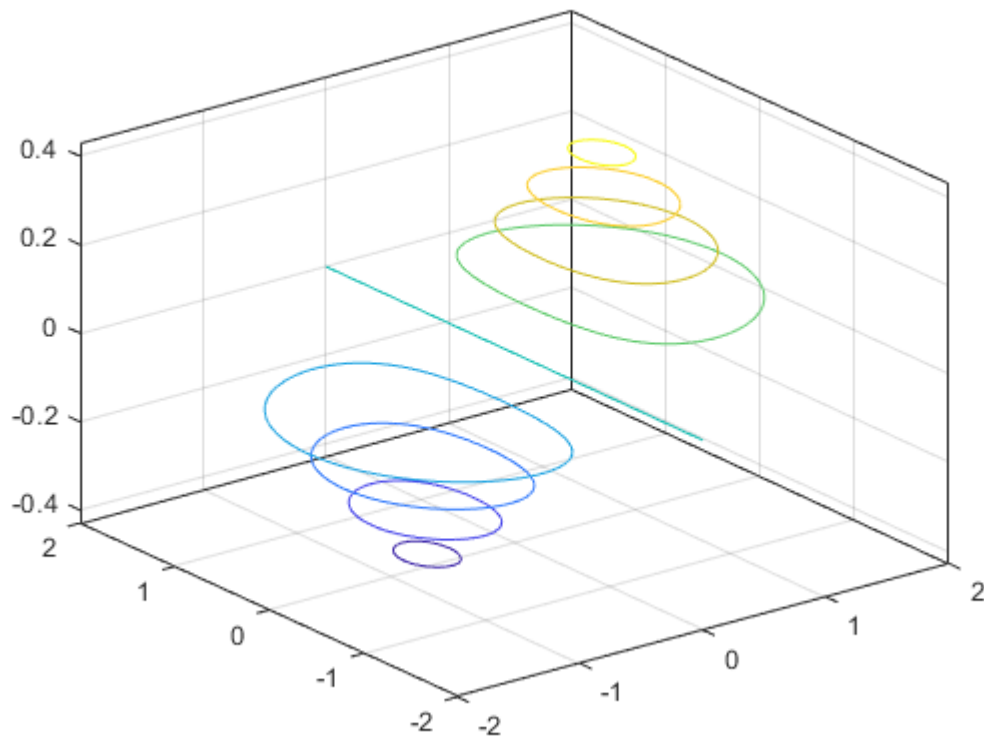
```
surf(X,Y,Z)
```



```
waterfall(X,Y,Z)
```



```
contour3(X,Y,Z)
```



Bài tập

Bài tập 1. Cho $x = [1 \ 4 \ 8]$, $y = [2 \ 1 \ 5]$ và $A = [3 \ 1 \ 6; \ 5 \ 2 \ 7]$. Xét xem dòng lệnh nào hợp lệ, dự đoán kết quả và thử lại bằng MATLAB:

1. $x + y$
2. $A - [x' \ y']$
3. $x + A$
4. $A - 3$
5. $x' + y$

% Sinh viên trình bày code lời giải cho bài tập 1 vào đây và sử dụng nút Run Section để thực thi.

Bài tập 2. Cho $A = [2 \ 7 \ 9 \ 7; \ 3 \ 1 \ 5 \ 6; \ 8 \ 1 \ 2 \ 5]$, dự đoán kết quả các lệnh, giải thích và thử lại bằng MATLAB:

1. A'
2. $\text{sum}(A)$
3. $\text{sum}(A')$
4. $\text{sum}(A, 2)$
5. $[[A; \text{sum}(A)][\text{sum}(A, 2); \text{sum}(A(:))]]$

% Sinh viên trình bày code lời giải cho bài tập 2 vào đây và sử dụng nút Run Section để thực thi.
% Lời giải thích ghi dưới dạng comment sử dụng dấu %

Bài tập 3. Nhập vào hai ma trận:

$$A = \begin{bmatrix} 2 & 1 & 0 \\ -2 & 5 & -1 \\ 3 & 4 & 9 \end{bmatrix}$$

$$B = \begin{bmatrix} 3 & 1 & 2 \\ -1 & 3 & -2 \\ 3 & 4 & 5 \end{bmatrix}$$

Thực hiện:

1. Tìm ma trận X sao cho $X^*B = A$.
2. Tìm ma trận X sao cho $X^*A = B$.
3. Xóa cột thứ 2 của ma trận A .
4. Thêm cột thứ nhất của ma trận B vào sau cột cuối của ma trận A .

% Sinh viên trình bày code lời giải cho bài tập 3 vào đây và sử dụng nút Run Section để thực thi.
% Lời giải thích ghi dưới dạng comment sử dụng dấu %

Bài tập 4. Cho ma trận $A = [2 \ 4 \ 1; \ 6 \ 7 \ 2; \ 3 \ 5 \ 9]$, viết các lệnh MATLAB:

1. Gán cho vector x dòng thứ nhất của A .
2. Gán cho y hai dòng còn lại của A .
3. Tìm phần tử lớn nhất và nhỏ nhất của ma trận A .
4. Tính tổng tất cả các phần tử của A .

% Sinh viên trình bày code lời giải cho bài tập 4 vào đây và sử dụng nút Run Section để thực thi.

% Lời giải thích ghi dưới dạng comment sử dụng dấu %

Bài tập 5. Tạo ma trận A vuông cỡ 4 có giá trị nguyên ngẫu nhiên nằm trong khoảng $[-10, 10]$, sau đó:

1. Cộng thêm 10 vào các phần tử của dòng 1 và dòng 2 của A , gán kết quả cho B .
2. Cộng thêm 10 vào các phần tử của cột 1 và cột 4 của A , gán kết quả cho C .
3. Tính nghịch đảo của các phần tử khác 0 của A , gán kết quả cho D .
4. Lấy căn bậc hai của mọi phần tử dương của A , gán kết quả cho E .

% Sinh viên trình bày code lời giải cho bài tập 5 vào đây và sử dụng nút Run Section để thực thi.
% Lời giải thích ghi dưới dạng comment sử dụng dấu %

Bài tập 6. Vẽ đồ thị của hàm số $y = x + \sin x$ theo biến x trong đó $x = -\pi : 0.1 : \pi$, sau đó thêm tiêu đề cho đồ thị và gán nhãn cho các trục tọa độ.

2. Vẽ đồ thị của hàm số $y = x^2 \sin x$ theo biến x trong đó $x = -\pi : 0.1 : \pi$, sau đó thêm tiêu đề cho đồ thị và gán nhãn cho các trục tọa độ.

% Sinh viên trình bày code lời giải cho bài tập 6 vào đây và sử dụng nút Run Section để thực thi.

Bài tập 7. Vẽ đồ thị các hàm $y = \sin x$, $y = \sin(x + \pi/2)$, $y = \sin(x + \pi/3)$ trên cùng một hệ trục tọa độ với vector $x = \text{linspace}(0, 2 * \pi)$.

% Sinh viên trình bày code lời giải cho bài tập 7 vào đây và sử dụng nút Run Section để thực thi.

Bài tập 8.

1. Nhập vào các dòng lệnh sau:

```
[X, Y] = meshgrid(-1 : .2 : 1);  
Z = X.^2 - Y.^2  
contour(Z)
```

Nhận xét về nhãn của các trục tọa độ. Hãy thử lại bằng lệnh `contour(X, Y, Z)`

2. Thử các lệnh sau và cho biết kết quả:

```
contourf (X, Y, Z)
contour (X, Y, Z, 10)
contour (X, Y, Z, 20)
contourf (X, Y, Z, 20)
```

% Sinh viên trình bày code lời giải cho bài tập 8 vào đây và sử dụng nút Run Section để thực thi.

Bài tập 9. Nhập vào các lệnh sau để định nghĩa và vẽ đồ thị hàm $g(x) = \exp(x)$:

```
syms x
g = exp(x)
ezplot(g)
```

Sau đó, điều chỉnh lại miền xác định bằng cách nhập: **ezplot**(g, [-2, 2]).

% Sinh viên trình bày code lời giải cho bài tập 9 vào đây và sử dụng nút Run Section để thực thi.

Bài tập 10. Nhập lệnh sau và xem kết quả:

```
syms x y
ezmesh(sin(x)*cos(y), [0, 10, 0, 10])
```

2. Kích vào **Tools**, sau đó kích vào **Rotate 3 D** . Trở vào đồ thị, dùng chuột trái giữ và quay ảnh theo ý bạn.
3. Sử dụng lệnh **ezmesh** vẽ đồ thị hàm số $f(x, y) = x^2 - y^2$ với $x, y \in [-2, 2]$.
4. Vẽ đồ thị hàm số $f(x, y) = \sin x^5 * \cos y$ sử dụng miền mặc định.

% Sinh viên trình bày code lời giải cho bài tập 10 vào đây và sử dụng nút Run Section để thực thi.

Tuần 4: Công cụ tính toán hình thức Symbolic

Nội dung bài thực hành:

1. Công cụ tính toán hình thức Symbolic

Sinh viên lưu file dưới theo định dạng "Tên không dấu + MSSV.mlx" và nộp bài trên nhóm teams của lớp ở mục assignments 4.

Mọi đóng góp xin liên hệ: tam.doduc@hust.edu.vn

Tóm tắt lý thuyết

Khai báo biến syms

Lệnh `sym` và `syms` cho phép ta xây dựng, biến đổi các số, biến và đối tượng thành Symbolic.

tên biến = sym (giá trị của biến)

syms danh sách biến Symbolic

```
x = sym('x', 'real')
```

```
x = x
```

```
assumptions x
```

```
ans = x ∈ ℝ
```

```
syms x clear
```

Warning: SYMS X CLEAR and SYMS('X','CLEAR') will not accept CLEAR in a future release. SYMS X clears assumptions by default.

```
assumptions x
```

```
ans =  
Empty sym: 1-by-0
```

```
syms a b c d % chú ý các biến syms xuất hiện ở workspace
```

Ta có thể thay giá trị số cho một biến Symbolic bằng cách sử dụng lệnh `subs` theo cấu trúc

subs(biểu thức Symbolic, tên biến Symbolic, giá trị số thay vào biến)

```
syms x  
f=x^2+3*x-2
```

```
f = x2 + 3 x - 2
```

```
subs(f,x,2)
```

```
ans = 8
```

Ứng dụng Symbolic trong giải tích

Để tính đạo hàm của một hàm số kiểu Symbolic ta sử dụng lệnh diff

diff(biểu thức sym, biến đạo hàm, cấp đạo hàm)

```
syms x y z
f = x^2+2*y^4*cos(z)
```

$$f = x^2 + 2 \cos(z) y^4$$

```
diff(f,y,3)
```

$$\text{ans} = 48 y \cos(z)$$

```
diff(f,z)
```

$$\text{ans} = -2 y^4 \sin(z)$$

```
diff(f,x,2)
```

$$\text{ans} = 2$$

Giới hạn

Để tính giới hạn của một hàm số kiểu Symbolic ta sử dụng lệnh limit

limit(biểu thức sym, biến tiến tới giới hạn, giới hạn của biến)

```
syms n
limit((1+1/n)^n,n,inf)
```

$$\text{ans} = e$$

```
syms x
limit(abs(x)/x,x,0,'left')
```

$$\text{ans} = -1$$

```
limit(abs(x)/x,x,0,'right')
```

$$\text{ans} = 1$$

Tích phân

Để tìm nguyên hàm và tích phân của một hàm số kiểu Symbolic ta sử dụng lệnh int

int(biểu thức sym, biến lấy tích phân, cận dưới, cận trên)

Nếu ta không khai báo các cận của tích phân thì MATLAB hiểu là cần tìm nguyên hàm.

```
syms x n
int(x^n,x)
```

ans =

$$\begin{cases} \log(x) & \text{if } n = -1 \\ \frac{x^{n+1}}{n+1} & \text{if } n \neq -1 \end{cases}$$

```
int(sin(x),x,0,1)
```

ans = 1 - cos(1)

Ta có thể sử dụng lệnh **int** để tính tích phân nhiều lớp. Để tính tích phân 2 lớp $I = \int_0^1 \int_{1-x}^{1-x^2} xy dy dx$ ta dùng các lệnh sau

```
syms x y
int(int(x*y,y,1-x,1-x^2),x,0,1)
```

ans =

$$\frac{1}{24}$$

Tính tổng hoặc chuỗi

Ta có thể tính các tổng Symbolic bằng cách sử dụng lệnh **symsum**

symsum(biểu thức Symbolic, biến chạy, cận dưới, cận trên)

Khi cận dưới hoặc cận trên bằng ∞ thì MATLAB hiểu là cần tính giá trị của chuỗi số hoặc chuỗi hàm.

Chuỗi

$$1 + \frac{1}{2^2} + \frac{1}{3^2} + \dots = \sum_{k=1}^{\infty} \frac{1}{k^2}$$

có tổng là $\pi^2/6$, trong khi chuỗi hàm

$$1 + x + x^2 + \dots = \sum_{k=0}^{\infty} x^k$$

có tổng là $\frac{1}{(1-x)}$, với $|x| < 1$. Các tổng trên có thể được tính bởi các lệnh

```
syms x k
s1=symsum(1/k^2,k,1,inf)
```

s1 =

$$\frac{\pi^2}{6}$$

```
assume(abs(x)<1)
s2 = symsum(x^k,k,0,inf)
```

s2 =

$$-\frac{1}{x-1}$$

Viết câu lệnh tính $\sum_{n=0}^{10} n = 0 + 1 + 2 + \dots + 10$

```
syms n
symsum(n,n,0,10)
```

ans = 55

Khai triển Taylor

Sử dụng lệnh taylor để khai triển taylor của một hàm số xung quanh điểm nào đó

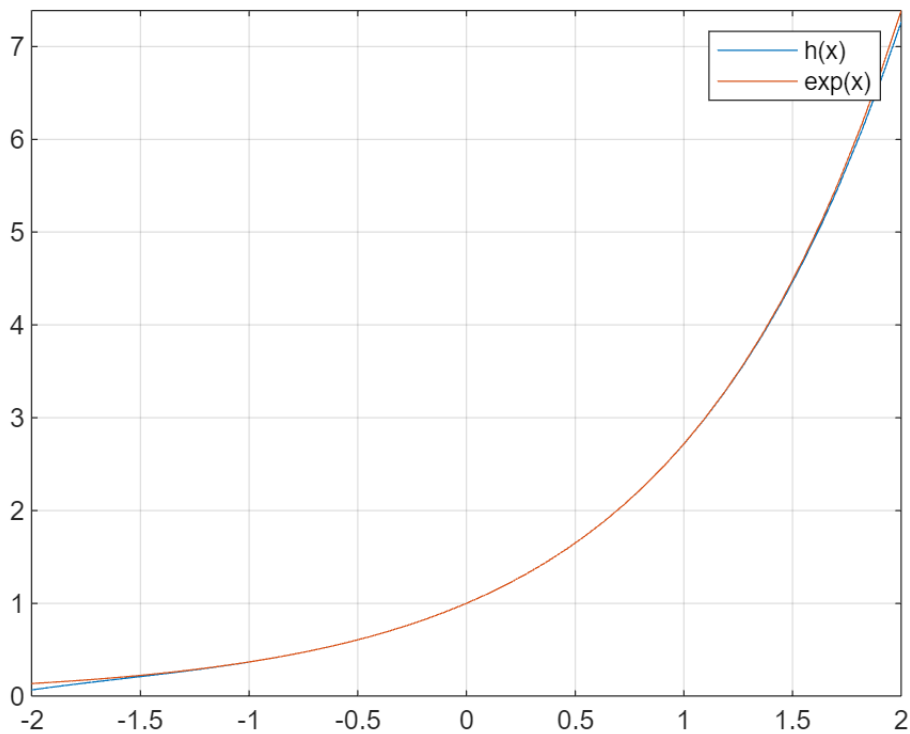
taylor(biểu thức sym, biến khai triển, điểm khai triển, 'Order', bậc khai triển)

```
syms x
h = taylor(exp(x),x,0,'Order',6)
```

h =

$$\frac{x^5}{120} + \frac{x^4}{24} + \frac{x^3}{6} + \frac{x^2}{2} + x + 1$$

```
fplot(h,[-2,2])
hold on
fplot(exp(x),[-2,2])
legend('h(x)','exp(x)')
grid on
```



Giải phương trình đại số

Nếu S là một biểu thức Symbolic thì lệnh **solve(S)** sẽ tìm giá trị của các biến Symbolic có trong S (có thể xác định bởi lệnh symvar) sao cho S=0. Cấu trúc của

solve(biểu thức Symbolic, biến giải nghiệm)

```
syms x clear
```

Warning: SYMS X CLEAR and SYMS('X','CLEAR') will not accept CLEAR in a future release. SYMS X clears assumptions by default.

```
r = solve(x^2-3*x-2==0)
```

r =

$$\begin{pmatrix} \frac{3}{2} - \frac{\sqrt{17}}{2} \\ \frac{\sqrt{17}}{2} + \frac{3}{2} \end{pmatrix}$$

```
syms a u v
```

```
eq1 = u^2-v^2==a^2
```

$$\text{eq1} = u^2 - v^2 = a^2$$

```
eq2 = u+v == 1
```

```
eq2 = u + v = 1
```

```
eq3 = a^2-2*a==3
```

```
eq3 = a2 - 2 a = 3
```

```
r1 = solve(eq1, eq2, eq3)
```

```
r1 = struct with fields:  
  a: [2×1 sym]  
  u: [2×1 sym]  
  v: [2×1 sym]
```

```
[r1.u,r1.v,r1.a]
```

```
ans =
```

```

$$\begin{pmatrix} 1 & 0 & -1 \\ 5 & -4 & 3 \end{pmatrix}$$

```

Giải phương trình vi phân

Hàm dsolve tìm các nghiệm SYMBOLIC của phương trình vi phân thường.

dsolve(phương trình 1,...,phương trình n)

trong đó **phương trình i** có kiểu Symbolic.

Ta có một số lưu ý khi viết các phương trình vi phân như sau:

- Phải khai báo các hàm cần tìm của hệ phương trình vi phân như là hàm trừu tượng theo các biến độc lập.
- Để biểu diễn đạo hàm của hàm số ta dùng lệnh **diff**. Biến độc lập mặc định là **t**. Tuy nhiên, có thể dùng biến độc lập khác bằng cách nhập nó như là thông số cuối của dsolve.
- Các điều kiện đầu có thể được mô tả như là các phương trình phụ, nếu không có điều kiện đầu, các nghiệm sẽ chứa các hằng số $C_1, C_2, \dots$.
- Các phương trình dạng $F(x, y, y', \dots, y^n) = 0$ ta chỉ cần nhập về trái; các phương trình dạng $F(x, y, y', \dots, y^n) = G(x, y, y', \dots, y^n)$ ta thay dấu $=$ bởi phép toán logic $==$.

Để giải phương trình $y' = 1 + y^2$ ta dùng câu lệnh

```
syms y(x)  
dsolve(diff(y)==1+y^2, subs(y,x,0)==1)
```

```
ans =
```

```
 $\tan\left(x + \frac{\pi}{4}\right)$ 
```

Xét hệ hai phương trình tuyến tính cấp 1

$$\begin{cases} f' = 3f + 4g. \\ g' = -4f + 3g. \end{cases}$$

Ta sử dụng lệnh như sau

```
syms f(t) g(t)
s = dsolve(diff(f)==3*f+4*g, diff(g)==-4*f+3*g)
```

```
s = struct with fields:
  g: C1*cos(4*t)*exp(3*t) - C2*sin(4*t)*exp(3*t)
  f: C2*cos(4*t)*exp(3*t) + C1*sin(4*t)*exp(3*t)
```

```
s.g
```

$$\text{ans} = C_1 \cos(4t) e^{3t} - C_2 \sin(4t) e^{3t}$$

```
s.f
```

$$\text{ans} = C_2 \cos(4t) e^{3t} + C_1 \sin(4t) e^{3t}$$

Bài tập

Bài tập 1. Tính các giới hạn sau:

$$1. \lim_{x \rightarrow +\infty} (\sin \sqrt{x+1} - \sin \sqrt{x})$$

$$2. \lim_{x \rightarrow \pi} \frac{1 - \cos x}{x^2}$$

$$3. \lim_{x \rightarrow a} \frac{\sin x - \sin a}{x - a}$$

$$4. \lim_{x \rightarrow 0} \frac{\log(\cos ax)}{\log(\cos bx)}$$

$$5. \lim_{x \rightarrow \pi} \frac{\sin 3x}{\sin 4x}$$

% Sinh viên trình bày code lời giải cho bài tập 1 vào đây và sử dụng nút Run Section để thực thi.

```
syms x
limit(sin(sqrt(x+1))-sin(sqrt(x)),x,+inf)
```

$$\text{ans} = 0$$

Bài tập 2. Cho $f(x) = x^5 + 3x^4 - 3x + 7$.

1. Tính đạo hàm $f'(x)$

2. Tính các giá trị $f(4)$, $f'(4)$ dùng lệnh subs

3. Sử dụng lệnh solve để tìm tất cả các điểm tới hạn của hàm $f(x)$..

% Sinh viên trình bày code lời giải cho bài tập 2 vào đây và sử dụng nút Run Section để thực thi.
% Lời giải thích ghi dưới dạng comment sử dụng dấu %

Bài tập 3. Tìm tất cả các nghiệm thực của phương trình $x = 4 \sin x$ bằng cách sử dụng lệnh fzero.

% Sinh viên trình bày code lời giải cho bài tập 3 vào đây và sử dụng nút Run Section để thực thi.
% Lời giải thích ghi dưới dạng comment sử dụng dấu %
help **fzero**

fzero Single-variable nonlinear zero finding.
X = **fzero**(FUN,X0) tries to find a zero of the function FUN near X0, if X0 is a scalar. It first finds an interval containing X0 where the function values of the interval endpoints differ in sign, then searches that interval for a zero. FUN is a function handle. FUN accepts real scalar input X and returns a real scalar function value F, evaluated at X. The value X returned by **fzero** is near a point where FUN changes sign (if FUN is continuous), or NaN if the search fails.

X = **fzero**(FUN,X0), where X0 is a vector of length 2, assumes X0 is a finite interval where the sign of FUN(X0(1)) differs from the sign of FUN(X0(2)). An error occurs if this is not true. Calling **fzero** with a finite interval guarantees **fzero** will return a value near a point where FUN changes sign.

X = **fzero**(FUN,X0), where X0 is a scalar value, uses X0 as a starting guess. **fzero** looks for an interval containing a sign change for FUN and containing X0. If no such interval is found, NaN is returned. In this case, the search terminates when the search interval is expanded until an Inf, NaN, or complex value is found. Note: if the option FunValCheck is 'on', then an error will occur if an NaN or complex value is found.

X = **fzero**(FUN,X0,OPTIONS) solves the equation with the default optimization parameters replaced by values in the structure OPTIONS, an argument created with the OPTIMSET function. See OPTIMSET for details. Used options are Display, TolX, FunValCheck, OutputFcn, and PlotFcns.

X = **fzero**(PROBLEM) finds the zero of a function defined in PROBLEM. PROBLEM is a structure with the function FUN in PROBLEM.objective, the start point in PROBLEM.x0, the options structure in PROBLEM.options, and solver name 'fzero' in PROBLEM.solver.

[X,FVAL] = **fzero**(FUN,...) returns the value of the function described in FUN, at X.

[X,FVAL,EXITFLAG] = **fzero**(...) returns an EXITFLAG that describes the exit condition. Possible values of EXITFLAG and the corresponding exit conditions are

- 1 **fzero** found a zero X.
- 1 Algorithm terminated by output function.
- 3 NaN or Inf function value encountered during search for an interval containing a sign change.

- 4 Complex function value encountered during search for an interval containing a sign change.
- 5 **fzero** may have converged to a singular point.
- 6 **fzero** can not detect a change in sign of the function.

[X,FVAL,EXITFLAG,OUTPUT] = **fzero**(...) returns a structure OUTPUT with the number of function evaluations in OUTPUT.funcCount, the algorithm name in OUTPUT.algorithm, the number of iterations to find an interval (if needed) in OUTPUT.intervaliterations, the number of zero-finding iterations in OUTPUT.iterations, and the exit message in OUTPUT.message.

Examples

FUN can be specified using @:

```
X = fzero(@sin,3)
```

returns pi.

```
X = fzero(@sin,3,optimset('Display','iter'))
```

returns pi, uses the default tolerance and displays iteration information.

FUN can be an anonymous function:

```
X = fzero(@(x) sin(3*x),2)
```

FUN can be a parameterized function. Use an anonymous function to capture the problem-dependent parameters:

```
myfun = @(x,c) cos(c*x); % The parameterized function.
```

```
c = 2; % The parameter.
```

```
X = fzero(@(x) myfun(x,c),0.1)
```

Limitations

```
X = fzero(@(x) abs(x)+1, 1)
```

returns NaN since this function does not change sign anywhere on the real axis (and does not have a zero as well).

```
X = fzero(@tan,2)
```

returns X near 1.5708 because the discontinuity of this function near the point X gives the appearance (numerically) that the function changes sign at X.

See also roots, fminbnd, function_handle.

Other uses of fzero

```
r = fzero(@(x)x-4*sin(x),1)
```

```
r = -3.4375e-20
```

Bài tập 4. Tính giá trị của các tích phân sau bằng cách sử dụng cả hai lệnh int và quadl:

$$1. \int_0^{\pi} \sqrt{1 + \sin^2 x} dx$$

$$2. \int_0^{\pi} \sqrt{1 + \sin^4 x} dx.$$

% Sinh viên trình bày code lời giải cho bài tập 4 vào đây và sử dụng nút Run Section để thực thi.

% Lời giải thích ghi dưới dạng comment sử dụng dấu %

help **quadl**

quadl Numerically evaluate integral, adaptive Lobatto quadrature.

Q = **quadl**(FUN,A,B) tries to approximate the integral of scalar-valued

function FUN from A to B to within an error of 1.e-6 using high order recursive adaptive quadrature. FUN is a function handle. The function Y=FUN(X) should accept a vector argument X and return a vector result Y, the integrand evaluated at each element of X.

Q = **quadl**(FUN,A,B,TOL) uses an absolute error tolerance of TOL instead of the default, which is 1.e-6. Larger values of TOL result in fewer function evaluations and faster computation, but less accurate results.

Q = **quadl**(FUN,A,B,TOL,TRACE) with non-zero TRACE shows the values of [fcnt a b-a Q] during the recursion. Use [] as a placeholder to obtain the default value of TOL.

[Q,FCNT] = **quadl**(...) returns the number of function evaluations.

Use array operators .*, ./ and .^ in the definition of FUN so that it can be evaluated with a vector argument.

quadl will be removed in a future release. Use **INTEGRAL** instead.

Example:

```
Q = quadl(@myfun,0,2);
where the file myfun.m defines the function:
%-----%
function y = myfun(x)
y = 1./(x.^3-2*x-5);
%-----%
```

or, use a parameter for the constant:

```
Q = quadl(@(x)myfun2(x,5),0,2);
where the file myfun2.m defines the function:
%-----%
function y = myfun2(x,c)
y = 1./(x.^3-2*x-c);
%-----%
```

Class support for inputs A, B, and the output of FUN:
float: double, single

See also **integral**, **integral2**, **integral3**, **quadgk**, **quad2d**, **trapz**, ...
function_handle.

```
quadl(@(x)sqrt(1+sin(x).^4),0,pi)
```

```
ans = 3.6525
```

Bài tập 5. Chứng minh rằng các hàm sau đây là nghiệm của phương trình vi phân tương ứng (sử dụng lệnh simplify nếu cần).

1. $y = 1 + e^{-t^2/2}, y' + ty = t$

2. $y = \frac{1}{t-3}, y' + y^2 = 0.$

3. $y = 10 - t^2/2, yy' + ty = 0.$

4. $y = \ln x, (2e^y - x)y' = 1.$ Nhắc lại, trong MATLAB, $\ln(x)$ là $\log(x)$.

5. $y = e^t \cos 2t, y'' - 2y' + 5y = 0.$

```
% Sinh viên trình bày code lời giải cho bài tập 5 vào đây và sử dụng nút Run  
Section để thực thi.  
% Lời giải thích ghi dưới dạng comment sử dụng dấu %
```

Bài tập 6. Xác định biến độc lập, sử dụng lệnh dsolve tìm nghiệm tổng quát của các phương trình. Tiếp đó, sử dụng lệnh subs để thay thế hằng số tích phân $C_1 = 2$. Cuối cùng, sử dụng lệnh ezplot để vẽ các hàm kết quả.

1. $y' + ty = t$
2. $yy' + ty = 0$
3. $y' + y^2 = 0$
4. $(x + y^2)y' = y$
5. $(2e^y - x)y' = 1$
6. $x(y' - y) = e^x$

```
% Sinh viên trình bày code lời giải cho bài tập 6 vào đây và sử dụng nút Run  
Section để thực thi.
```

Bài tập 7. Xác định biến độc lập, sử dụng lệnh dsolve tìm nghiệm của các bài toán giá trị ban đầu. Sử dụng lệnh ezplot để vẽ các hàm kết quả trên các khoảng tương ứng.

1. $y' + ty = t, y(0) = -1, [-4, 4]$
2. $(2e^y - x)y' = 1, y(0) = 0, [-5, 5]$
3. $y' + y^2 = 0, y(0) = 2, [0, 5]$
4. $yy' + ty = 0, y(1) = 4, [-4, 4]$
5. $(x + y^2)y' = y, y(0) = 4, [-4, 6]$
6. $x(y' - y) = e^x, y(1) = 4e, [0.001, 1].$

```
% Sinh viên trình bày code lời giải cho bài tập 7 vào đây và sử dụng nút Run  
Section để thực thi.
```

Bài tập 8. Chứng tỏ rằng:

1. Hàm số $z = y \ln(x^2 - y^2)$ thỏa mãn phương trình $\frac{1}{x} \frac{\partial z}{\partial x} + \frac{1}{y} \frac{\partial z}{\partial y} = \frac{z}{y^2} \dots$
2. Hàm số $z = y^{y/x} \sin \frac{y}{x}$ thỏa mãn phương trình $x^2 \frac{\partial z}{\partial x} + xy \frac{\partial z}{\partial y} = yz.$

% Sinh viên trình bày code lời giải cho bài tập 8 vào đây và sử dụng nút Run Section để thực thi.

Bài tập 9. Tính các tích phân bội

1. $\int_0^1 \int_{x^2}^1 (x+y) dy dx$
2. $\int_{-5}^5 \int_{-\sqrt{25-x^2}}^{\sqrt{25-x^2}} \int_{\sqrt{x^2+y^2}}^5 dz dy dx$
3. $\int_{\pi/2}^{\pi} \int_0^{x^2} \frac{1}{x} \cos \frac{y}{x} dy dx$
4. $\int_0^2 \int_{x^2}^x y^2 x dy dx$
5. $\int_0^1 \int_{-1}^2 \int_1^3 (x+y^2+z^3) dz dy dx$
6. $\int_0^6 \int_0^z \int_0^x xy dz dy dx$
7. $\int_1^4 \int_0^{\sqrt{y}} e^{x/\sqrt{y}} dx dy$

% Sinh viên trình bày code lời giải cho bài tập 9 vào đây và sử dụng nút Run Section để thực thi.

Bài tập 10. Tính tổng của các chuỗi hội tụ:

1. $\sum_{n=1}^{\infty} \frac{1}{n^2}$
2. $\sum_{n=1}^{\infty} \frac{1}{2^n}$
3. $\sum_{n=1}^{\infty} \frac{1}{n^4}$
4. $\sum_{n=1}^{\infty} x^n, |x| < 1..$

% Sinh viên trình bày code lời giải cho bài tập 10 vào đây và sử dụng nút Run Section để thực thi.

Tuần 5: Lập trình với MATLAB

Nội dung bài thực hành:

1. Thủ tục và hàm trong MATLAB
2. Xuất dữ liệu và nhập dữ liệu
3. Cấu trúc điều khiển
4. Cấu trúc rẽ nhánh
5. Cấu trúc lặp

Sinh viên lưu file dưới theo định dạng "Tên không dấu + MSSV.mlx" và nộp bài trên nhóm teams của lớp ở mục assignments 2.

Mọi đóng góp xin liên hệ: tam.doduc@hust.edu.vn

Tóm tắt lý thuyết

Thủ tục là một khối các câu lệnh được thực hiện một cách tuần tự khi ta gọi tên thủ tục đó. Các biến được sử dụng là một phần của không gian làm việc (**Workspace**).

Thủ tục vẽ đồ thị hàm số $\sin(t)$

`% vẽ đồ thị hàm số sin(t) trong khoảng (-pi;pi).`

```
t = -pi : 0.01 : pi;
```

```
y = sin(t);
```

```
plot(t, y, 'r--');
```

Lưu file trên với tên **PlotSinx.m** vào thư mục hiện hành (**Current Folder**). Để thực thi thủ tục **PlotSinx** ở cửa sổ lệnh (**Command Window**) ta gõ tên của m-file.

Hàm là chương trình sử dụng các tham số đầu vào/đầu ra để kết hợp chúng với các hàm khác và các lệnh trong **Command Window**. Các biến được sử dụng trong hàm là các biến địa phương (các biến này không xuất hiện trong **Workspace**).

Dòng đầu tiên của hàm "m-file" có dạng

function [outArgs]=funName(inArgs)

trong đó outArgs là danh sách các biến đầu ra, được đặt trong []

- Các biến trong outArgs được cách nhau bởi dấu ",".
- [] là tùy chọn nếu chỉ có 1 tham số đầu ra.
- Hàm mà không có outArgs vẫn là hợp lệ.

và danh sách các biến đầu vào inArgs được đặt trong ()

- Các biến trong inArgs được cách nhau bởi dấu ",".

- Hàm mà không có inArgs vẫn là hợp lệ.
- Có thể kiểm tra tính hợp lệ của tên hàm bằng cách dùng lệnh **isvarname funName**.

Các toán tử quan hệ và logic

Bảng các toán tử quan hệ và logic:

Toán tử	Ý nghĩa
<	<
<=	≤
>	>
>=	≥
==	=
~=	≠
&&	and
&	and (tác động được lên vectơ)
	or
	or (tác động được lên vectơ)
~	not

Các cấu trúc rẽ nhánh **if**:

if <biểu thức điều kiện>

khối lệnh

end

```
vanToc = input('Nhập vận tốc gió\n')
```

```
vanToc =  
45
```

```
if vanToc<=38  
    disp('Đây là áp thấp')  
end
```

Các cấu trúc rẽ nhánh **if ... else**:

if <biểu thức điều kiện>

khối lệnh 1

else

khối lệnh 2

end

```
if vanToc<=38  
    disp('Đây là áp thấp')  
else  
    disp('Đây là bão')
```

end

Đây là bão

Các cấu trúc rẽ nhánh **if ... elseif:**

if <biểu thức điều kiện 1>

khối lệnh 1

elseif <biểu thức điều kiện 2>

khối lệnh 2

else

khối lệnh 3

end

```
if vanToc<=38
    disp('Đây là áp thấp.')
elseif (vanToc>38)&&(vanToc<=73)
    disp('Đây là bão.')
else
    disp('Đây là siêu bão.')
end
```

Đây là bão.

Câu lệnh **switch** thường được sử dụng khi tập các giá trị của các biến kiểm tra là rời rạc. Khi gặp lệnh này, MATLAB so sánh giá trị của biểu thức sau **switch** với các giá trị sau case, trùng với giá trị nào thì thực hiện khối lệnh tương ứng.

Cú pháp câu lệnh switch:

switch <biểu thức>

case <giá trị 1>

<khối lệnh 1>

case <giá trị 2>

<khối lệnh 2>

....

otherwise

<khối lệnh 3>

end

```
num = input('Nhập vào một số nguyên\n')
```



```
num =  
1
```

```
switch num  
    case 1  
        disp("Bạn nhập vào số 1.")  
    case -1  
        disp("Bạn nhập vào số -1.")  
    case 0  
        disp("Bạn nhập vào số 0")  
    otherwise  
        disp("Bạn nhập vào số khác -1, 0, 1")  
  
end
```

Bạn nhập vào số 1.

Cấu trúc lặp với **for**:

for <index> = [tập giá trị]

<khối lệnh>

end

```
v = 1:10;  
s = 0;  
for index = 1:length(v)  
    s = s+index;  
end
```

Cấu trúc lặp với **while**:

while <biểu thức logic>

<khối lệnh>

end

```
index = 1;  
s = 0;  
while index <= length(v)  
    s = s+v(index);  
    index = index + 1;  
end  
fprintf('Tổng 10 số tự nhiên đầu tiên là %d',s)
```

tổng 10 số tự nhiên đầu tiên là 55

Bài tập

Bài tập 1.

1. Tạo một thủ tục MATLAB với tên 'test1.m' thực hiện các công việc sau:

- Xóa tất cả các biến trong bộ nhớ
- Tạo một vector hàng $x = 1 : 0.1 : 10$;
- Tính $y = \sin x - \cos x$
- Vẽ đồ thị của y theo đối số x .

2. Viết thêm các chú thích và lưu chương trình của bạn vào trong thư mục làm việc. Sau đó

- Kiểm tra và gỡ lỗi nếu có
- Hiển thị các chú thích bằng lệnh `>>help test1`

% Sinh viên trình bày code lời giải cho bài tập 1 vào đây và sử dụng nút Run Section để thực thi.

Bài tập 2. 1. Tạo một hàm tên 'test2.m' với biến đầu vào (input) x và biến đầu ra y tính toán và hiển thị trên màn hình.

a. Tính $y = \sin x - \cos x$

b. Vẽ đồ thị của hàm y theo đối số x .

2. Viết thêm các chú thích và lưu chương trình của bạn vào trong thư mục làm việc. Sau đó:

a. Kiểm tra chương trình với $x = 1 : 0.1 : 10$ và gỡ lỗi nếu có

b. Hiển thị các chú thích bằng lệnh `>>help test2`

3. Giải thích sự khác nhau giữa thủ tục và hàm.

% Sinh viên trình bày code lời giải cho bài tập 2 vào đây và sử dụng nút Run Section để thực thi.
% Lời giải thích ghi dưới dạng comment sử dụng dấu %

Bài tập 3. Tính gần đúng số π sử dụng kết quả sau: $\frac{\pi^2}{6} = \sum_{n=1}^{\infty} \frac{1}{n^2}$. Với sai số là $1e-8$ thì cần ít nhất bao nhiêu số hạng?

% Sinh viên trình bày code lời giải cho bài tập 3 vào đây và sử dụng nút Run Section để thực thi.
% Lời giải thích ghi dưới dạng comment sử dụng dấu %

Bài tập 4. Có một thuật toán khác để tính gần đúng số π như sau:

1. Đặt $a = 1, b = 1/\sqrt{2}, t = 1/4, x = 1$.

2. Lặp lại quy trình dưới đây cho đến khi $|b - a| < \varepsilon$, ε là sai số cho trước:

$$y = a$$

$$a = (a + b)/2$$

$$b = \sqrt{b \cdot y}$$

$$t = t - x \cdot (y - a)^2$$

$$x = 2 \cdot x$$

3. Từ các giá trị a, b, t , ước lượng giá trị π bởi công thức

$$\pi_{\text{est}} = ((a + b)^2)/(4 \cdot t)$$

Hãy viết chương trình MATLAB thể hiện thuật toán trên. Cần bao nhiêu bước lặp để thu được sai số là $1e-8$, $1e-12$? So sánh với thuật toán trong bài tập 3.

```
% Sinh viên trình bày code lời giải cho bài tập 4 vào đây và sử dụng nút Run
Section để thực thi.
% Lời giải thích ghi dưới dạng comment sử dụng dấu %
```

Bài tập 5. Viết các chương trình tính $n!, (2n)!!, (2n + 1)!!$.

```
% Sinh viên trình bày code lời giải cho bài tập 5 vào đây và sử dụng nút Run
Section để thực thi.
% Lời giải thích ghi dưới dạng comment sử dụng dấu %
```

Bài tập 6. Dãy Fibonacci là dãy số được xác định như sau:

$$F_1 = F_2 = 1, F_n = F_{n-1} + F_{n-2}, n \geq 3.$$

1. Tính 10 số hạng đầu tiên của dãy Fibonacci

2. Với 50 số hạng đầu tiên của dãy, tính tỷ số $\frac{F_n}{F_{n-1}}$.

Người ta chứng minh được rằng dãy tỷ số này tiến tới tỷ lệ vàng $\rho = \frac{1 + \sqrt{5}}{2}$.

Kiểm định điều này qua kết quả của bạn.

```
% Sinh viên trình bày code lời giải cho bài tập 6 vào đây và sử dụng nút Run
Section để thực thi.
```

Bài tập 7. Đa thức Legendre được định nghĩa theo công thức truy hồi sau:

$$(n+1)P_{n+1}(x) - (2n+1)xP_n(x) + nP_{n-1}(x) = 0.$$

với $P_0(x) = 1, P_1(x) = x, P_2(x) = \frac{3x^2 - 1}{2}$. Lập chương trình tính đa thức Legendre bậc n , kết quả lưu dưới dạng vector hệ số.

% Sinh viên trình bày code lời giải cho bài tập 7 vào đây và sử dụng nút Run Section để thực thi.

Bài tập 8. Đa thức Chebyshev được định nghĩa theo công thức truy hồi như sau:

$$T_{n+1}(x) = 2xT_n(x) - T_{n-1}(x).$$

với $T_0(x) = 1, T_1(x) = x$, lập chương trình tính đa thức Chebyshev bậc n , kết quả lưu dưới dạng vector hệ số.

% Sinh viên trình bày code lời giải cho bài tập 8 vào đây và sử dụng nút Run Section để thực thi.

Bài tập 9. Viết chương trình tính tiền điện của một gia đình biết rằng giá điện thay đổi phụ thuộc vào số điện tiêu thụ n theo quy định sau:

- Mỗi số điện (Kwh) có giá 500đ nếu $n < 100$
- Mỗi số điện có giá 700 đ nếu $100 \leq n < 200$
- Mỗi số điện có giá 1000 đ nếu $200 \leq n \leq 500$
- Mỗi số điện có giá 1200 đ nếu $n > 500$.

Đồng thời nếu $n < 0$ thì in ra không hợp lệ, nếu $n > 500$ thì in ra tổng số tiền kèm theo câu 'Hạn chế sử dụng điện.'

% Sinh viên trình bày code lời giải cho bài tập 9 vào đây và sử dụng nút Run Section để thực thi.

Bài tập 10. Viết chương trình giải phương trình bậc hai $ax^2 + bx + c = 0$. Áp dụng để giải các phương trình $x^2 - 3x + 2 = 0; x^2 + 2x - 1 = 0$. Kiểm tra lại kết quả bằng cách tính tay hoặc dùng hàm roots của MATLAB.

% Sinh viên trình bày code lời giải cho bài tập 10 vào đây và sử dụng nút Run Section để thực thi.

Bài tập 11. Viết chương trình với input là một số nguyên dương n , output là hai số S_1, S_2 trong đó

$$S_1 = \sum_{k=1}^n k, S_2 = \sum_{k=1}^n \frac{1}{k^3}. \text{ Áp dụng tính với } n = 2012, \text{ sử dụng } \textit{format long} \text{ để in ra kết quả.}$$

% Sinh viên trình bày code lời giải cho bài tập 11 vào đây và sử dụng nút Run Section để thực thi.

Bài tập 12. Viết chương trình tính căn bậc hai của a theo sai số cho trước ε theo công thức lặp Newton:

- Chọn $x_1 = \frac{a}{2}$.

- Sử dụng công thức lặp: $x_{n+1} = \frac{1}{2} \left(x_n + \frac{a}{x_n} \right)$.

- Điều kiện dừng: $\frac{|x_{n+1} - x_n|}{|x_n|} < \varepsilon$.

% Sinh viên trình bày code lời giải cho bài tập 12 vào đây và sử dụng nút Run Section để thực thi.

Bài tập 13. Viết chương trình tính gần đúng căn bậc ba của a với sai số cho trước ε theo công thức lặp Newton:

- Chọn $x_1 = \frac{a}{3}$.

- Sử dụng công thức lặp $x_{n+1} = \frac{1}{3} \left(2x_n + \frac{a}{x_n^2} \right)$.

- Điều kiện dừng: $\frac{|x_{n+1} - x_n|}{|x_n|} < \varepsilon$.

% Sinh viên trình bày code lời giải cho bài tập 13 vào đây và sử dụng nút Run Section để thực thi.

Bài tập 14. Viết chương trình MATLAB tính tổng $\sum_{k=1}^n \frac{1}{k^3}$ bằng cách sử dụng mã vô hướng và mã vector hóa. So sánh thời gian chạy của hai cách trên.

% Sinh viên trình bày code lời giải cho bài tập 14 vào đây và sử dụng nút Run Section để thực thi.

Tuần 6: Phương pháp chia đôi

Nội dung bài thực hành:

1. Lập trình tìm khoảng cách ly nghiệm.
2. Lập trình cho phương pháp chia đôi.

Tóm tắt lý thuyết

Nội dung phương pháp

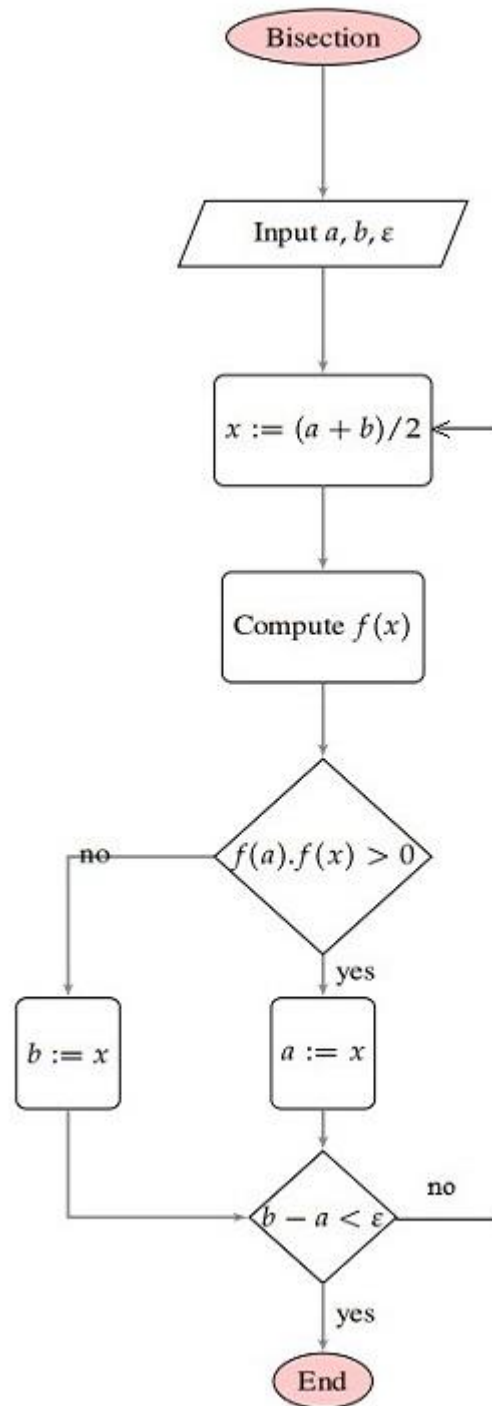
Điều kiện thực hiện

- (a, b) là khoảng cách ly nghiệm của phương trình.
- $f(x)$ liên tục trên (a, b) .
- $f(a)f(b) < 0$.

Thực hiện chia đôi:

1. Chia lần thứ 1: $x_1 = \frac{a+b}{2}$.
2. Kiểm tra nếu $f(x_1) = 0$ thì kết luận x_1 là nghiệm của phương trình.
3. Kiểm tra nếu $f(x_1)f(a) < 0$ thì (a, x_1) là khoảng cách ly nghiệm mới, đặt là (a_1, b_1) . Nếu trái lại thì (x_1, b) là khoảng cách ly nghiệm mới, đặt là (a_1, b_1) .
4. Thực hiện chia lần 2 tương tự lần 1 với $x_2 = \frac{a_1+b_1}{2}$ cho khoảng (a_1, b_1) .
5. Tiếp tục thực hiện chia cho đến khi thu được kết quả như mong muốn.

Sơ đồ khối:



Code gợi ý phương pháp chia đôi

```

function mid = ChiaDoi(fun, start, finish, eps)
% fun là hàm vế trái của phương trình, fun = @(x)biểu thức
%(start,finish) là khoảng phân ly
%eps là sai số mong muốn
mid = NaN;
while finish - start > eps
    mid = (start + finish) / 2;
    if feval(fun, mid) == 0
        return
    end
end
  
```

```

    if feval(fun,start)*feval(fun, mid)<0
        finish = mid;
    else
        start = mid;
    end
end
end
end

```

Ví dụ: Tìm nghiệm phương trình: $2^x + x - 4 = 0$ bằng phương pháp chia đôi.

Câu lệnh MATLAB gọi hàm chia đôi: `fun = 2x + x - 4; start = 1, finish = 2; eps = 1e - 3.`

```
r = chiaDoi(@(x)2.^x+x-4,1,2,1e-3)
```

Giải ví dụ trên sử dụng thủ tục MATLAB

Đặt

```
fun = @(x)2.^x+x-4
```

fun = function_handle with value:

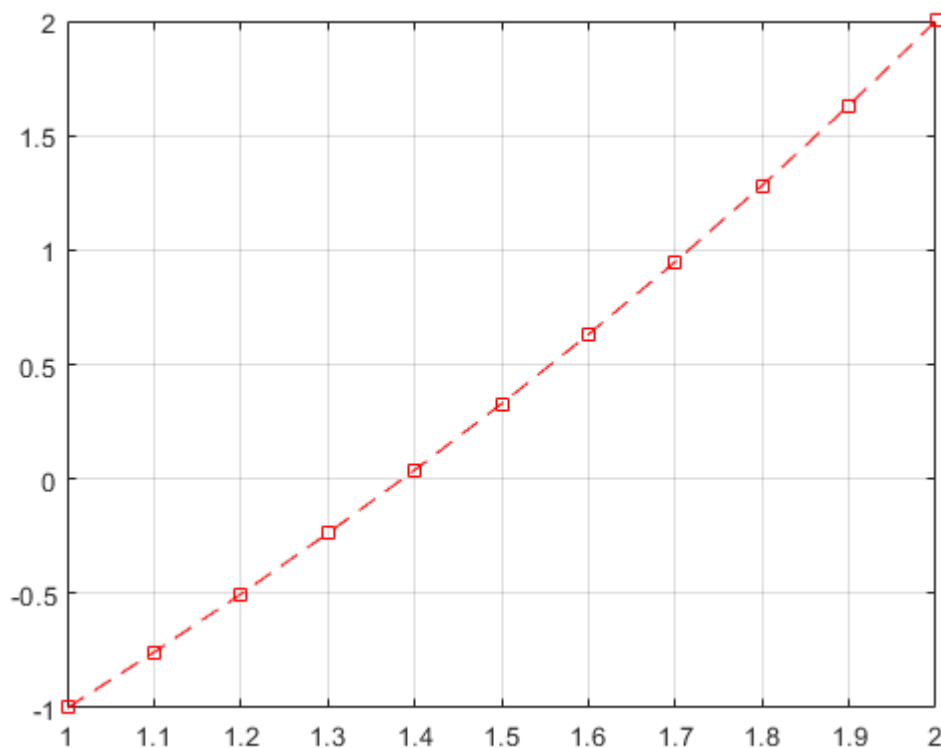
```
@(x)2.^x+x-4
```

Vẽ đồ thị thấy khoảng (1,2) là một khoảng phân ly nghiệm.

```

x = 1:.1:2;
plot(x,fun(x),'--rs')
grid on

```



Thực hiện phương pháp chia đôi:

```
format long;  
% gán giá trị  
start = 1; finish =2; eps = 1e-5;  
xn = NaN;  
% bắt đầu phương pháp chia đôi  
while finish - start>eps  
    xn=(start+finish)/2  
    if feval(fun,xn) == 0  
        return  
    end  
    if feval(fun,start)*feval(fun, xn)<0  
        finish = xn;  
    else  
        start = xn;  
    end  
end
```

```
xn =  
1.500000000000000  
xn =  
1.250000000000000  
xn =  
1.375000000000000  
xn =  
1.437500000000000  
xn =  
1.406250000000000  
xn =  
1.390625000000000  
xn =  
1.382812500000000  
xn =  
1.386718750000000  
xn =  
1.384765625000000  
xn =  
1.385742187500000  
xn =  
1.386230468750000  
xn =  
1.385986328125000  
xn =  
1.386108398437500  
xn =  
1.386169433593750  
xn =  
1.386138916015625  
xn =  
1.386154174804688  
xn =  
1.386161804199219
```

Bài tập

Bài tập 1.

Viết chương trình MATLAB kiểm tra (a, b) có phải là khoảng cách li nghiệm của phương trình $f(x) = 0$ hay không? Kiểm tra chương trình với phương trình $x^4 - 4x + 2 = 0$ và khoảng $(0, 1)$.

Bài tập 2.

Viết chương trình MATLAB giải gần đúng phương trình $f(x) = 0$ tìm nghiệm trong khoảng (a, b) sử dụng phương pháp chia đôi với n lần lặp. Kiểm tra chương trình với phương trình $x^4 - 4x + 2 = 0$ và khoảng $(0, 1)$ và $n = 10$.

Tuần 7.1: Phương pháp dây cung

Nội dung thực hành

1. Viết chương trình MATLAB cho phương pháp dây cung

Mọi đóng góp xin liên hệ: tam.doduc@hust.edu.vn

Tóm tắt lý thuyết

Điều kiện thực hiện:

- (a, b) là khoảng cách ly nghiệm của phương trình.
- f', f'' xác định dấu không đổi trên $[a, b]$.
- Chọn đúng điểm mốc d và xuất phát ban đầu x_0 .

Thực hiện phương pháp dây cung

Bước 1: Kiểm tra điều kiện thực hiện.

Bước 2: Chọn xấp xỉ đầu x_0 sao cho $f(x_0)f''(x_0) < 0$ và tham số d sao cho $f(d)f''(d) > 0$.

Bước 3: Xây dựng dãy nghiệm xấp xỉ

$$x_{n+1} = x_n - \frac{(d - x_n)f(x_n)}{f(d) - f(x_n)}, \quad n = 0, 1, 2, \dots$$

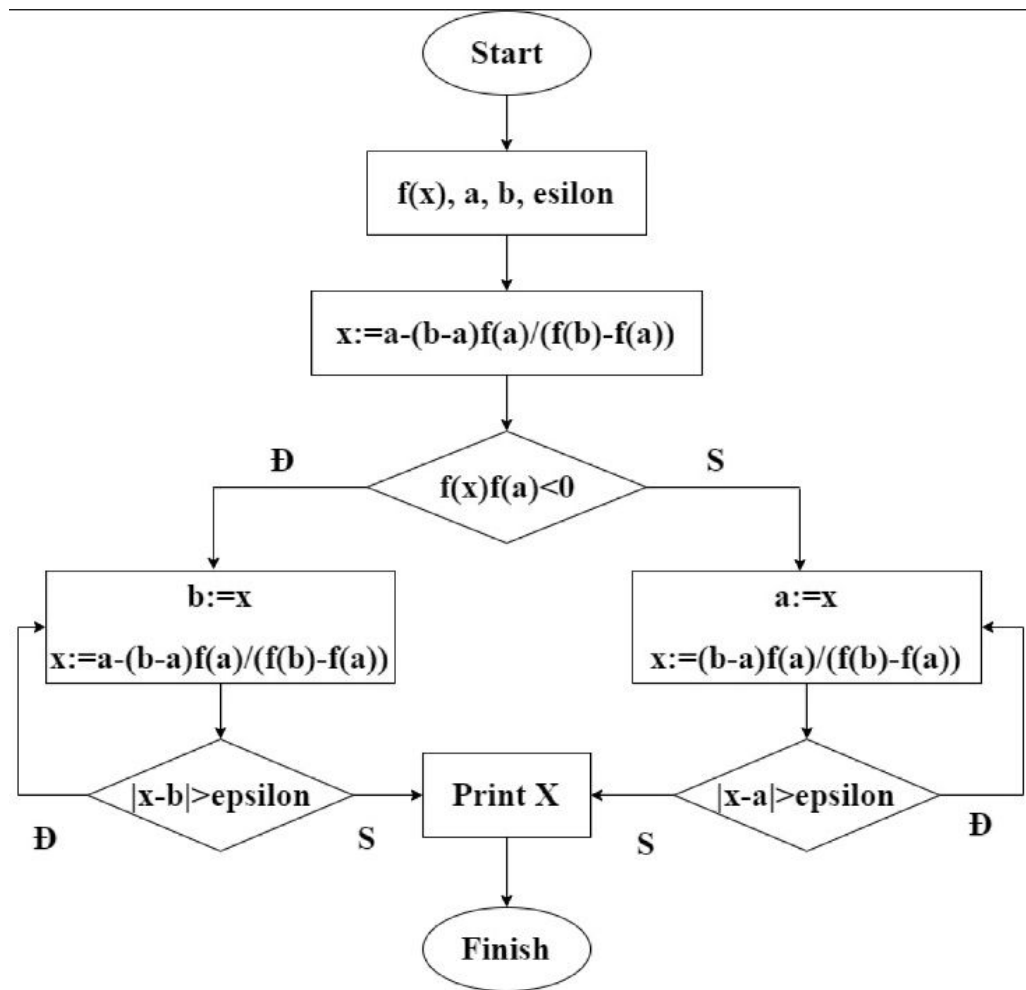
Sai số

Sai số của nghiệm gần đúng x_n lấy từ dãy lặp ở bước 3

$$|x_n - x^*| \leq \frac{M_1 - m_1}{m_1} |x_n - x_{n-1}|,$$

trong đó $M_1 = \max_{[a,b]} |f'(x)|$ và $0 < m_1 = \min_{[a,b]} |f'(x)|$.

Sơ đồ khối:



Code gợi ý phương pháp dây cung:

```

function x1 = DayCung(fun, a, b, eps)
% fun là vế trái của phương trình, fun dạng symbolic
% (a,b) là khoảng phân ly nghiệm
% eps là sai số mong muốn
x1 = NaN;
df = diff(fun);
d2f = diff(fun,2);
check = false;
% kiểm tra điều kiện hội tụ
n = ceil((b-a)/0.1)+1;
t = linspace(a,b,n);
Df = double(subs(df,t));
D2f = double(subs(d2f,t));
if ((all(Df>0)) && (all(D2f>0))) || ((all(Df<0)) && (all(D2f<0))) || ((all(Df>0)) && (all(D2f<0))) ||
((all(Df<0)) && (all(D2f>0)))
    check = true;
end
if check == false
    disp('Phương pháp tiếp tuyến không hội tụ')
    return
end
% tính tham số m1 và M2
m1 = double(min(abs(Df)));
M1 = double(max(abs(Df)));

```

```

% xác định x0
x0 = a; d = b;
if subs(fun,b)*sign(D2f(end))<0
    x0 = b;
    d = a;
end

saiSo = eps + 1;
while saiSo > eps
    x1 = x0 - (d - x0)*subs(fun,x0)/(subs(fun,d) - subs(fun,x0));
    x1 = double(x1);
    saiSo = (M1 - m1)/m1*abs(x1-x0);
    x0 = x1;
end
end

```

Ví dụ: Tìm nghiệm của phương trình $x^3 - x - 1 = 0$ sử dụng phương pháp tiếp tuyến sao cho sai số đạt $1e-10$.

Câu lệnh MATLAB gọi hàm DayCung: $\text{fun} = x^3 - x - 1; a = 1; b = 1.5; \text{eps} = 1e-10$.

```

syms x
fun = x^3-x-1;
a = 1;
b = 1.5;
eps = 1e-10;
r = DayCung(fun, a, b, eps)

```

Giải ví dụ trên sử dụng thủ tục MATLAB

Khoảng phân ly nghiệm là $(1, 1.5)$.

Đặt $f(x) = x^3 - x - 1$. Ta có $f'(x) = 3x^2 - 1 > 2 \quad \forall x \in [1, 1.5]$. Lại có $f''(x) = 6x \geq 6 \quad \forall x \in [1, 1.5]$. Chọn $d = 1.5$ do $f(1.5)f''(1.5) = 7.8750 > 0$. Chọn $x_0 = 1$.

```

syms x
f = x^3-x-1

```

$$f = x^3 - x - 1$$

```
df = diff(fun)
```

$$df = 3x^2 - 1$$

```
d2f = diff(fun,2)
```

$$d2f = 6x$$

```

n = ceil((b-a)/0.1)+1;
t = linspace(a,b,n);
Df = double(subs(df,t));
D2f = double(subs(d2f,t));

```

```
m1 = double(min(abs(Df)))
```

```
m1 =  
2
```

```
M1 = double(max(abs(Df)))
```

```
M1 =  
5.750000000000000
```

Do đó phương pháp dây cung hội tụ với phương trình đã cho.

```
d = 1; x0 = 1.5;  
double(subs(f,1.5)*subs(diff(f,2),1.5))
```

```
ans =  
7.875000000000000
```

```
subs(f,1.5)
```

```
ans =  
 $\frac{7}{8}$ 
```

Xây dựng dãy nghiệm xấp xỉ theo công thức $x_{n+1} = x_n - \frac{(1.5 - x_n)(x_n^3 - x_n - 1)}{7/8 - (x_n^3 - x_n - 1)}$, $n = 0, 1, 2, \dots$

```
eps = 1e-10;  
saiSo = eps + 1
```

```
saiSo =  
1.000000000100000
```

```
while saiSo > eps  
    x1 = x0 - (d - x0)*subs(fun,x0)/(subs(fun,d) - subs(fun,x0));  
    x1 = double(x1)  
    saiSo = (M1 - m1)/m1*abs(x1-x0);  
    x0 = x1;  
end
```

```
x1 =  
1.266666666666667  
x1 =  
1.348297213622291  
x1 =  
1.315835759592585  
x1 =  
1.328163715000862  
x1 =  
1.323396170095247  
x1 =  
1.325227197943357  
x1 =  
1.324522090811171  
x1 =
```

```

1.324793340670588
x1 =
1.324688951474695
x1 =
1.324729119044584
x1 =
1.324713662195298
x1 =
1.324719609998764
x1 =
1.324717321261139
x1 =
1.324718201973234
x1 =
1.324717863072531
x1 =
1.324717993482492
x1 =
1.324717943300356
x1 =
1.324717962610588
x1 =
1.324717955179954
x1 =
1.324717958039284
x1 =
1.324717956939006
x1 =
1.324717957362396
x1 =
1.324717957199474
x1 =
1.324717957262167
x1 =
1.324717957238043

```

Vậy nghiệm gần đúng đạt sai số $1e-10$ là 1.324717957238043.

Bài tập

Bài tập 1.

Viết chương trình MATLAB giải gần đúng phương trình $f(x) = 0$ tìm nghiệm trong khoảng (a, b) sử dụng phương pháp dây cung với n lần lặp. Kiểm tra chương trình với phương trình $2^x - 5x + \sin(x) = 0$ và khoảng cách li $(0, 0.5)$ và $n = 4$.

Tuần 7.2: Phương pháp tiếp tuyến

Nội dung thực hành

1. Viết chương trình MATLAB cho phương pháp tiếp tuyến

Mọi đóng góp xin liên hệ: tam.doduc@hust.edu.vn

Tóm tắt lý thuyết

Nội dung phương pháp

Điều kiện thực hiện:

- (a, b) là khoảng cách ly nghiệm của phương trình.
- f', f'' xác định dấu không đổi trên $[a, b]$.
- Chọn đúng điểm mốc d và xuất phát ban đầu x_0 .

Thực hiện phương pháp dây cung

Bước 1: Kiểm tra điều kiện thực hiện.

Bước 2: Chọn xấp xỉ đầu x_0 sao cho $f(x_0)f''(x_0) > 0$.

Bước 3: Xây dựng dãy nghiệm xấp xỉ

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}, \quad n = 0, 1, 2, \dots$$

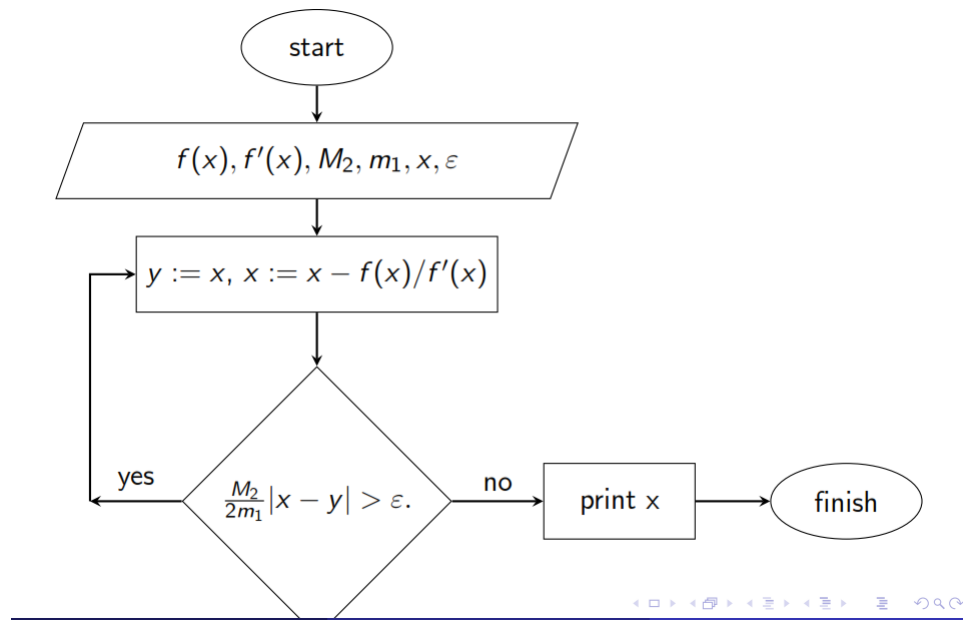
Sai số

Sai số của nghiệm gần đúng x_n lấy từ dãy lặp ở bước 3

$$|x_n - x^*| \leq \frac{M_2}{2m_1} (x_n - x_{n-1})^2,$$

trong đó $M_2 = \max_{[a,b]} |f''(x)|$ và $0 < m_1 = \min_{[a,b]} |f'(x)|$.

Sơ đồ khối:



Code gợi ý phương pháp tiếp tuyến:

```

function x1 = TiejTuyen(fun, a, b, eps)
% fun là vế trái của phương trình, fun dạng symbolic
% (a,b) là khoảng phân ly nghiệm
% eps là sai số mong muốn
x1 = NaN;
df = diff(fun);
d2f = diff(fun,2);
check = false;
% kiểm tra điều kiện hội tụ
n = ceil((b-a)/0.1)+1;
t = linspace(a,b,n);
Df = double(subs(df,t));
D2f = double(subs(d2f,t));
if ((all(Df>0)) && (all(D2f>0))) || ((all(Df<0)) && (all(D2f<0))) || ((all(Df>0)) && (all(D2f<0))) ||
((all(Df<0)) && (all(D2f>0)))
    check = true
end
if check == false
    disp('Phương pháp tiếp tuyến không hội tụ')
    return
end
% tính tham số m1 và M2
m1 = double(min(abs(Df)));
M2 = double(max(abs(D2f)));
% xác định x0
x0 = a;
if subs(fun,b)*sign(D2f(end))>0
    x0 = b;
end
saiSo = eps + 1;
while saiSo > eps
    x1 = x0 - subs(fun,x0)/subs(df,x0);
    x1 = double(x1);
    saiSo = M2/(2*m1)*(x1-x0)^2;
    x0 = x1;
end

```

```
end  
end
```

Ví dụ: Tìm nghiệm của phương trình $x^3 - x - 1 = 0$ sử dụng phương pháp tiếp tuyến sao cho sai số đạt $1e-10$.

Câu lệnh MATLAB gọi hàm TiejTuyen: $\text{fun} = x^3 - x - 1; a = 1; b = 1.5; \text{eps} = 1e - 10.$

```
syms x  
fun = x^3-x-1;  
a = 1;  
b = 1.5;  
eps = 1e-10;  
r = TiejTuyen(fun, a, b, eps)
```

Giải ví dụ trên sử dụng thủ tục MATLAB

Khoảng phân ly nghiệm là $(1, 1.5)$.

Đặt $f(x) = x^3 - x - 1$. Ta có $f'(x) = 3x^2 - 1 > 2 \quad \forall x \in [1, 1.5]$. Lại có $f''(x) = 6x \geq 6 \quad \forall x \in [1, 1.5]$. Chọn $x_0 = 1.5$ do $f(1.5)f''(1.5) = 7.8750 > 0$. Do đó phương pháp tiếp tuyến hội tụ với phương trình đã cho.

```
syms x  
f = x^3-x-1
```

$$f = x^3 - x - 1$$

```
double(subs(f,1.5)*subs(diff(f,2),1.5))
```

```
ans =  
7.875000000000000
```

Xây dựng dãy nghiệm xấp xỉ theo công thức $x_{n+1} = x_n - \frac{x_n^3 - x_n - 1}{3x_n^2 - 1}, \quad n = 0, 1, 2, 3, 4.$

```
x = zeros(1,6);  
x(1) = 1.5;  
for i = 1:5  
    x(i+1) = x(i)-(x(i)^3-x(i)-1)/(3*x(i)^2-1);  
end  
disp ('nghiem tìm được sau 5 lần lặp là:')
```

nghiem tìm được sau 5 lần lặp là:

```
disp(x')
```

```
1.500000000000000  
1.347826086956522  
1.325200398950907  
1.324718173999054  
1.324717957244790  
1.324717957244746
```

Vậy nghiệm gần đúng sau 5 lần lặp là $x_5 = 1.324717957244746$. Sai số của nghiệm x_5 là

$$|x_5 - x^*| \leq \frac{M_2}{2m_1}(x_5 - x_4)^2,$$

trong đó $M_2 = \max_{[1,1.5]}|6x| = 6 * 1.5 = 9$ và $m_1 = \min_{[1,1.5]}|3x^2 - 1| = 3 * 1 - 1 = 2$. Do đó

$$|x_5 - x^*| \leq \frac{9}{2 * 2}(1.324717957244746 - 1.324717957244790)^2 = 9.842127113302013e - 14.$$

Bài tập

Bài tập 1.

Viết chương trình MATLAB giải gần đúng phương trình $f(x) = 0$ tìm nghiệm trong khoảng (a, b) sử dụng phương pháp tiếp tuyến với n lần lặp. Kiểm tra chương trình với phương trình $2^x - 5x + \sin(x) = 0$ và khoảng cách li $(0, 0.5)$ và $n = 4$.

Tuần 8: Phương pháp lặp đơn

Nội dung thực hành

1. Viết chương trình MATLAB cho phương pháp lặp đơn

Mọi đóng góp xin liên hệ: tam.doduc@hust.edu.vn

Tóm tắt lý thuyết

Nội dung phương pháp

Bước 1: Biến đổi phương trình $f(x) = 0$ về dạng $x = g(x)$, với $g(x)$ là một hàm số thỏa mãn tính chất: tồn tại số $q \in (0, 1)$ sao cho

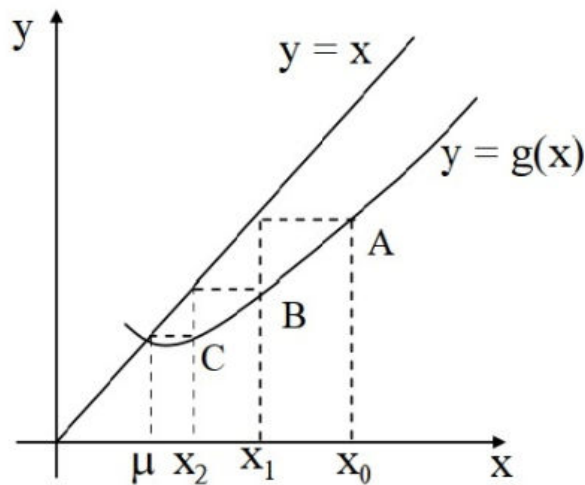
$$|g'(x)| \leq q \quad \forall x \in [a, b].$$

Bước 2: Chọn xấp xỉ ban đầu bất kỳ $x_0 \in [a, b]$.

Bước 3: Xây dựng dãy nghiệm xấp xỉ theo công thức

$$x_{n+1} = g(x_n) \quad n = 0, 1, 2, \dots$$

Ý tưởng hình học



Hình a

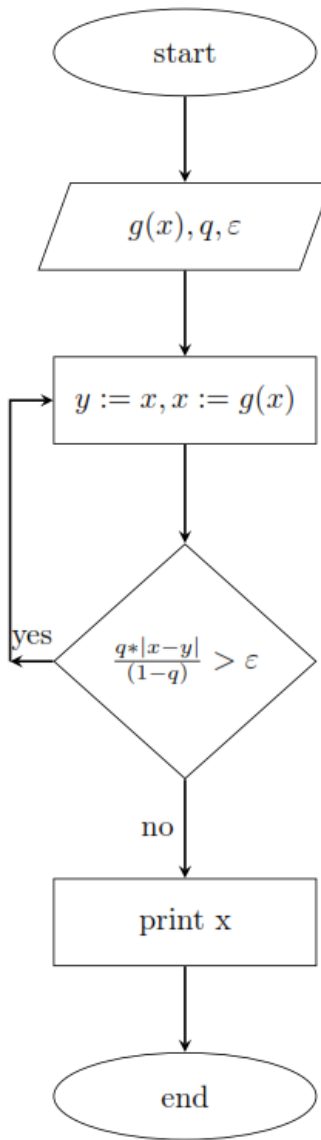
Sai số

Ta có công thức đánh giá sai số của nghiệm gần đúng x_n lấy ra từ dãy ở bước 3.

$$|x_n - x^*| \leq \frac{q}{1-q} |x_n - x_{n-1}|$$

$$|x_n - x^*| \leq \frac{q^n}{1-q} |x_1 - x_0|$$

Sơ đồ khối:



Ví dụ:

Tìm nghiệm của phương trình $x^3 - x - 1 = 0$ sử dụng phương pháp lặp đơn để đạt được sai số $\varepsilon = 0.5e - 5$.

Gợi ý viết **hàm m-file (không dùng Run để chạy được Code example, phải viết code vào m-file riêng):**

```

function x1 = LapDon(fun, x0, a, b, eps)
%fun là hàm lặp dạng symbolic
%x0 là xấp xỉ ban đầu
%(a,b) là khoảng phân ly nghiệm
%eps là sai số mong muốn
    dfun = diff(fun);
% tìm hệ số co q
  
```

```

        h = 0.1; n = ceil((b-a)/0.1);
        t = linspace(a, b, n+1);
        y = subs(dfun, t);
        q = max(abs(y));
        q = double(q);
    saiSo = eps+1;
    while saiSo > eps
        x1 = double(subs(fun,x0));
        saiSo = q/(1-q)*abs(x1-x0);
        x0 = x1;
    end
end

```

Ví dụ gọi hàm LapDon cho phương trình $x^3 - x - 1 = 0$

```

syms x
% khai báo hàm lặp
fun = (1+x)^(1/3)
% gán các giá trị
a = 1, b = 1.5, x0 = 1, eps = 1.5,
r = LapDon(fun, x0, a, b, eps)

```

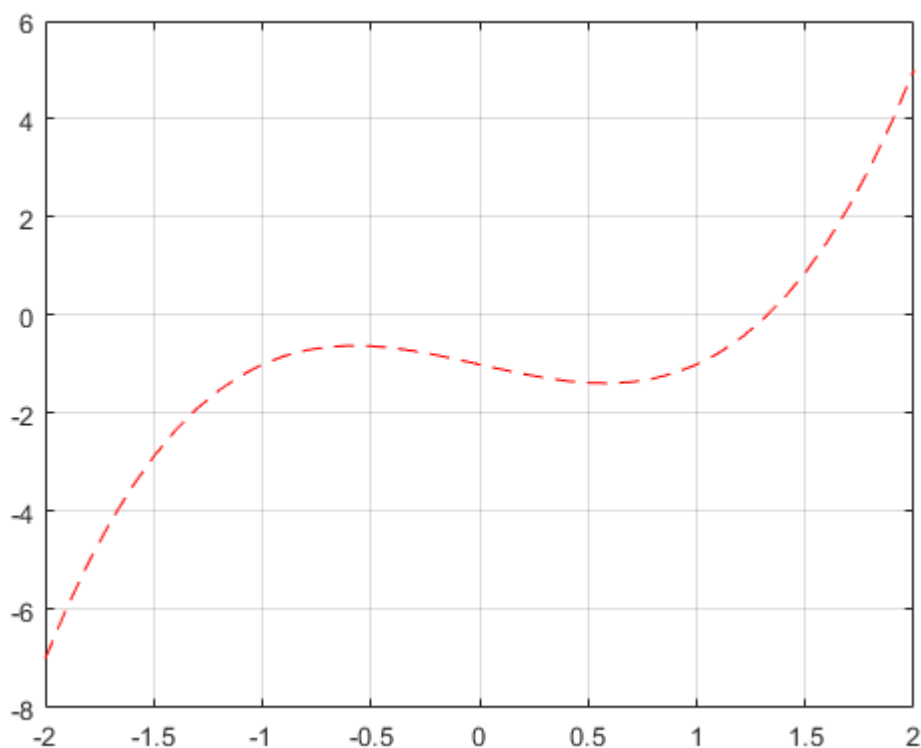
Giải ví dụ trên sử dụng thủ tục MATLAB

Bước 1: Tìm khoảng phân ly nghiệm

```

x = -2:.1:2;
y = x.^3-x-1;
plot(x,y,'-r')
grid on

```



Chứng minh $(1, 1.5)$ là khoảng phân ly nghiệm. Đặt $f(x) = x^3 - x - 1$. Ta có $f'(x) = 3x^2 - 1 > 2 \quad \forall x \in [1, 1.5]$. Suy ra hàm số $f(x)$ đồng biến trên $[1, 1.5]$. Hơn nữa, $f(1)f(1.5) = -0.8750 < 0$. Do đó, khoảng $(1, 1.5)$ là khoảng phân ly nghiệm của phương trình đã cho.

```
syms x
f = x^3-x-1
```

$$f = x^3 - x - 1$$

```
double(subs(f,1)*subs(f,1.5))
```

```
ans =
-0.8750000000000000
```

Bước 2: Biến đổi tương đương $x^3 - x - 1 \Leftrightarrow x = (x + 1)^{1/3}$. Chọn $g(x) = (x + 1)^{1/3}$.

Ta có $g'(x) = \frac{1}{3(x+1)^{2/3}}$. Dễ thấy $|g'(x)| \leq g'(1) = 0.2100 = q < 1 \quad \forall x \in [1, 1.5]$. Do đó phương pháp lặp đơn hội tụ.

```
syms x
g = (x+1)^(1/3)
```

$$g = (x + 1)^{1/3}$$

```
dg = diff(g)
```

```
dg =  
1  
3 (x + 1)2/3
```

```
double(subs(dg,1))
```

```
ans =  
0.209986841649146
```

Bước 3: Chọn xấp xỉ $x_0 = 1$.

Bước 4: Xây dựng dãy nghiệm xấp xỉ theo công thức

$$x_{n+1} = (x_n + 1)^{1/3}, n = 0, 1, 2, \dots$$

```
x0 = 1
```

```
x0 =  
1
```

```
epsilon = 0.5e-5;  
saiSo = 1;  
while saiSo > epsilon  
    x1 = (x0+1)^(1/3)  
    saiSo = 0.21/(1-0.21)*abs(x1-x0);  
    x0=x1;  
end
```

```
x1 =  
1.259921049894873  
x1 =  
1.312293836683289  
x1 =  
1.322353819138825  
x1 =  
1.324268744551578  
x1 =  
1.324632625250920  
x1 =  
1.324701748510359  
x1 =  
1.324714878440950
```

```
format long  
x1
```

```
x1 =  
1.324714878440950
```

Vậy nghiệm gần đúng cần tìm là $x = 1.324714878440950$.

Bài tập

Bài tập 1.

Viết chương trình MATLAB giải gần đúng phương trình $f(x) = 0$ tìm nghiệm trong khoảng (a, b) sử dụng phương pháp lặp đơn với n lần lặp. Kiểm tra chương trình với phương trình $x^3 + 3x^2 - 1 = 0$ và khoảng cách li $(-3, -2.5)$ và $n = 7$.

Phương pháp Gauss và Gauss - Jordan giải hệ phương trình $AX = b$.

Phương pháp khử Gauss:

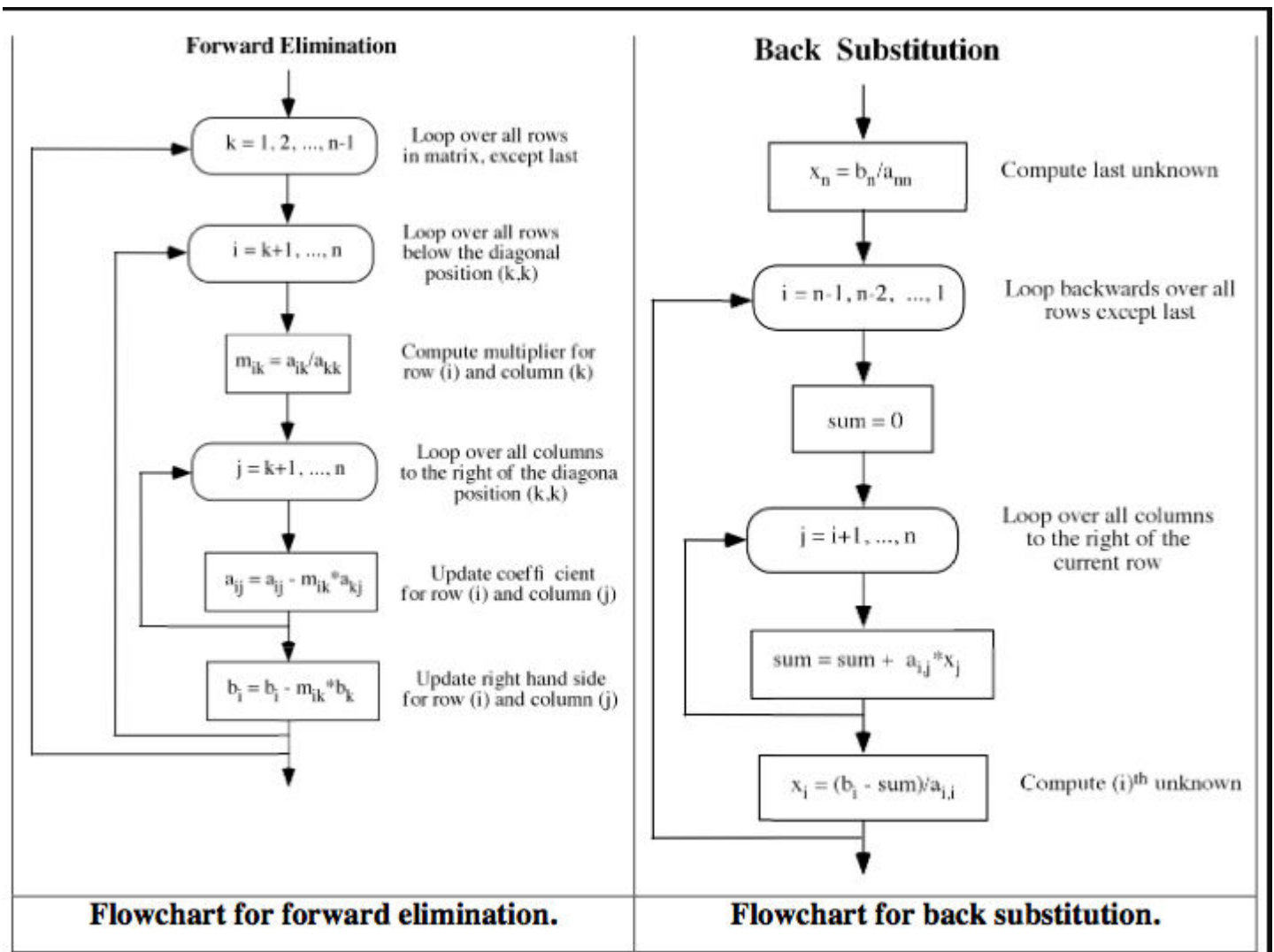
Bước Gauss thuận: Để khử biến x_k khỏi các phương trình thứ $k + 1, k + 2, \dots, n$ ta thực hiện

$$H_i \leftarrow H_i - \frac{a_{ik}}{a_{kk}} H_k, \quad i = \overline{k+1, n}.$$

Bước Gauss nghịch: Sau khi có hệ bậc thang ta tìm $x_n = b_n/a_{nn}$. Các nghiệm x_k tìm theo công thức

$$x_k = \frac{1}{a_{kk}} (b_k - \sum_{j=k+1}^n a_{kj} x_j), \quad k = \overline{n-1, 1}.$$

Sơ đồ khối:



Gợi ý Code cho phương pháp khử Gauss

```
function B = GaussThuan(A, b)
```

```

n = size(A, 1);
B=[A, b];
for k = 1: n-1
    if B(k, k) == 0
        disp('loi chia cho 0')
        return
    else
        for i = k+1:n
            B(i,:) = B(i, :) - B(k, :)*B(i,k)/B(k, k);
        end
    end
end
end
end

```

```

function nghiem = Gaussnghich(B)
%B la ma tran tam giac tren
n = size(B, 1);
nghiem = zeros(n, 1);
for k = n:-1:1
    if k == n
        nghiem(n) = B(n,n+1)/B(n,n);
    else
        S = B(k,n+1)-B(k,1:n)*nghiem;
        nghiem(k) = S/B(k,k);
    end
end
end

```

Ví dụ: Giải hệ phương trình

$$\begin{bmatrix} 1 & 3 & 1 \\ 2 & 4 & -1 \\ 1 & 8 & 3 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 3 \\ 5 \\ 6 \end{bmatrix}$$

Câu lệnh MATLAB gọi hàm GaussThuan và GaussNghich:

```

A = [1 3 1; 2 4 -1; 1 8 3];
b = [3; 5; 6];
B = GaussThuan(A, b)
X = GaussNghich(B)

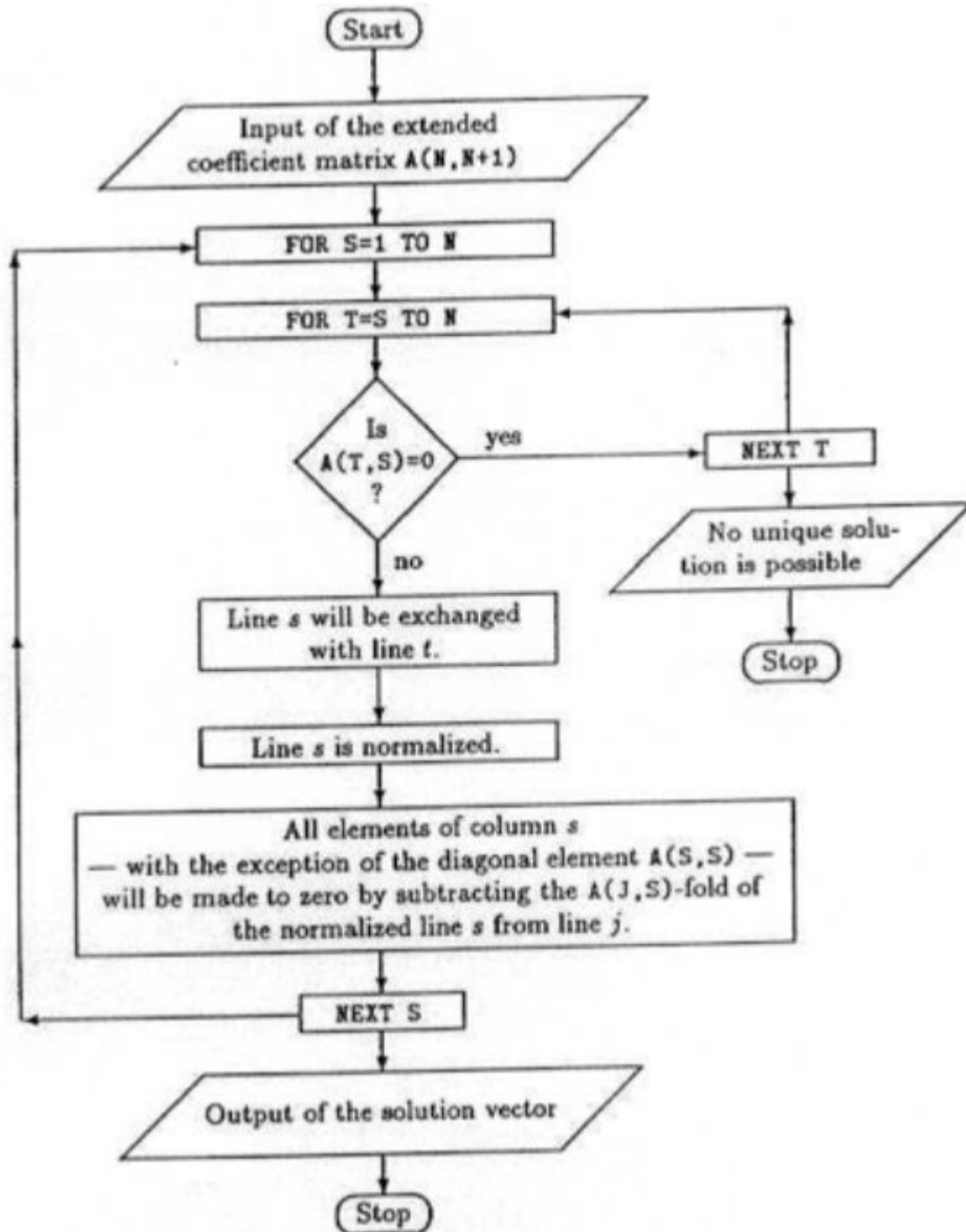
```

Kết quả chạy chương trình:

$$B = \begin{bmatrix} 1 & 3 & 1 & 3 \\ 0 & -2 & -3 & -1 \\ 0 & 0 & -5.5 & 0.5 \end{bmatrix}; X = \begin{bmatrix} 1.1818 \\ 0.6364 \\ -0.0909 \end{bmatrix}$$

Ghi chú: Code gợi ý chưa hoạt động được trong trường hợp ma trận A suy biến. Sinh viên cải tiến code để xử lý trường hợp này.

Phương pháp Gauss - Jordan:



Code gợi ý phương pháp Gauss-Jordan:

```

function [A, jb] = GJ(A, tol)

[m,n] = size(A);

% Loop over the entire matrix.
i = 1;
j = 1;
%create empty row vector
jb = zeros(1,0);

```

```

while i <= m && j <= n
    % Find value and index of largest element in the remainder of column j.
    [p, k] = max(abs(A(i:m,j)));
    k = k+i-1;
    if p < tol
        % The column is negligible, zero it out.
        A(i:m,j) = 0;
        j = j + 1;
    else
        % Remember column index
        jb = [jb j]; %#ok<AGROW>
        % Swap i-th and k-th rows.
        A([i k],j:n) = A([k i],j:n);
        % Divide the pivot row by the pivot element.
        A(i,j:n) = A(i,j:n)./A(i,j);
        % Subtract multiples of the pivot row from all the other rows.
        for k = [1:i-1 i+1:m]
            A(k,j:n) = A(k,j:n) - A(k,j).*A(i,j:n);
        end
        i = i + 1;
        j = j + 1;
    end
end
end

```

Ví dụ: Giải hệ phương trình

$$\begin{bmatrix} 1 & 3 & 1 \\ 2 & 4 & -1 \\ 1 & 8 & 3 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 3 \\ 5 \\ 6 \end{bmatrix}$$

Câu lệnh MATLAB gọi hàm GJ

```

A = [1 3 1; 2 4 -1; 1 8 3];
b = [3; 5; 6];
A = [A, b];
tol = 1e-5;
[A, jb] = GJ(A, tol)

```

Kết quả chạy chương trình:

$$A = \begin{bmatrix} 1 & 0 & 0 & 1.1818 \\ 0 & 1 & 0 & 0.6364 \\ 0 & 0 & 1 & -0.0909 \end{bmatrix} \Rightarrow X = \begin{bmatrix} 1.1818 \\ 0.6364 \\ -0.0909 \end{bmatrix}; \text{jb} = [1 \quad 2 \quad 3]$$

Ghi chú: Ở đây véc tơ jb đưa ra vị trí các ẩn độc lập.

Tuần 9: Phương pháp Gauss và Gauss - Jordan giải hệ phương trình $AX = b$.

Nội dung thực hành

1. Viết chương trình MATLAB cho phương pháp khử Gauss.
2. Viết chương trình MATLAB cho phương pháp Gauss-Jordan.

Mọi đóng góp xin liên hệ: tam.doduc@hust.edu.vn

Tóm tắt lý thuyết

Phương pháp khử Gauss:

Bước Gauss thuận: Để khử biến x_k khỏi các phương trình thứ $k+1, k+2, \dots, n$ ta thực hiện

$$H_i \leftarrow H_i - \frac{a_{ik}}{a_{kk}} H_k, \quad i = \overline{k+1, n}.$$

Bước Gauss nghịch: Sau khi có hệ bậc thang ta tìm $x_n = b_n/a_{nn}$. Các nghiệm x_k tìm theo công thức

$$x_k = \frac{1}{a_{kk}} (b_k - \sum_{j=k+1}^n a_{kj} x_j), \quad k = \overline{n-1, 1}.$$

Gợi ý Code cho phương pháp khử Gauss

```
function B = GaussThuan(A, b)
n = size(A, 1);
B=[A, b];
for k = 1: n-1
    if B(k, k) == 0
        disp('loi chia cho 0')
        return
    else
        for i = k+1:n
            B(i,:) = B(i, :) - B(k, :)*B(i,k)/B(k, k);
        end
    end
end
end
```

```
function nghiem = Gaussnghich(B)
%B la ma tran tam giac tren
n = size(B, 1);
nghiem = zeros(n, 1);
for k = n:-1:1
    if k == n
        nghiem(n) = B(n,n+1)/B(n,n);
    else
        S = B(k,n+1)-B(k,1:n)*nghiem;
```

```

        nghiem(k) = S/B(k,k);
    end
end

```

Ví dụ: Giải hệ phương trình

$$\begin{bmatrix} 1 & 3 & 1 \\ 2 & 4 & -1 \\ 1 & 8 & 3 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 3 \\ 5 \\ 6 \end{bmatrix}$$

Câu lệnh MATLAB gọi hàm GaussThuan và GaussNghich:

```

A = [1 3 1; 2 4 -1; 1 8 3];
b = [3; 5; 6];
B = GaussThuan(A, b)
X = GaussNghich(B)

```

Kết quả chạy chương trình:

$$B = \begin{bmatrix} 1 & 3 & 1 & 3 \\ 0 & -2 & -3 & -1 \\ 0 & 0 & -5.5 & 0.5 \end{bmatrix}; X = \begin{bmatrix} 1.1818 \\ 0.6364 \\ -0.0909 \end{bmatrix}$$

Ghi chú: Code gợi ý chưa hoạt động được trong trường hợp ma trận A suy biến. Sinh viên cải tiến code để xử lý trường hợp này.

Code gợi ý phương pháp Gauss-Jordan:

```

function [A, jb] = GJ(A, tol)

[m,n] = size(A);

% Loop over the entire matrix.
i = 1;
j = 1;
%create empty row vector
jb = zeros(1,0);
while i <= m && j <= n
    % Find value and index of largest element in the remainder of column j.
    [p, k] = max(abs(A(i:m,j)));
    k = k+i-1;
    if p < tol
        % The column is negligible, zero it out.
        A(i:m,j) = 0;
        j = j + 1;
    else
        % Remember column index
        jb = [jb j]; %#ok<AGROW>
        % Swap i-th and k-th rows.
        A([i k],j:n) = A([k i],j:n);
        % Divide the pivot row by the pivot element.
        A(i,j:n) = A(i,j:n)./A(i,j);
        % Subtract multiples of the pivot row from all the other rows.
        for k = [1:i-1 i+1:m]

```

```

        A(k,j:n) = A(k,j:n) - A(k,j).*A(i,j:n);
    end
    i = i + 1;
    j = j + 1;
end
end

```

Ví dụ: Giải hệ phương trình

$$\begin{bmatrix} 1 & 3 & 1 \\ 2 & 4 & -1 \\ 1 & 8 & 3 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 3 \\ 5 \\ 6 \end{bmatrix}$$

Câu lệnh MATLAB gọi hàm GJ

```

A = [1 3 1; 2 4 -1; 1 8 3];
b = [3; 5; 6];
A = [A, b];
tol = 1e-5;
[A, jb] = GJ(A, tol)

```

Kết quả chạy chương trình:

$$A = \begin{bmatrix} 1 & 0 & 0 & 1.1818 \\ 0 & 1 & 0 & 0.6364 \\ 0 & 0 & 1 & -0.0909 \end{bmatrix} \Rightarrow X = \begin{bmatrix} 1.1818 \\ 0.6364 \\ -0.0909 \end{bmatrix}; \text{jb} = [1 \quad 2 \quad 3]$$

Ghi chú: Ở đây vectơ jb đưa ra vị trí các ẩn độc lập.

Bài tập

Bài tập 1.

Viết thuật toán cho phương pháp khử Gauss, từ đó viết chương trình MATLAB cho thuật toán vừa xây dựng. Kiểm tra chương trình với hệ sau

$$\begin{bmatrix} 1 & 1 & -3 & 3 \\ 1 & -2 & -1 & 0 \\ 0 & 1 & 1 & 2 \\ 2 & -3 & 2 & 0 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} = \begin{bmatrix} 6 \\ 2 \\ 12 \\ 6 \end{bmatrix}.$$

Bài tập 2.

Viết thuật toán cho phương pháp Gauss-Jordan, từ đó viết chương trình MATLAB cho thuật toán vừa xây dựng. Kiểm tra chương trình với hệ sau

$$\begin{bmatrix} 5 & 1 & -3 & 7 \\ 1 & 0 & -1 & 8 \\ 6 & 12 & 7 & 2 \\ 2 & -3 & 2 & 0 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} = \begin{bmatrix} 1 \\ -2 \\ 4 \\ 6 \end{bmatrix}.$$

Tuần 9: Phương pháp lặp đơn

Nội dung thực hành

1. Viết chương trình MATLAB cho phương pháp lặp đơn

Mọi đóng góp xin liên hệ: tam.doduc@hust.edu.vn

Tóm tắt lý thuyết

Nội dung phương pháp

Bước 1: Biến đổi phương trình $f(x) = 0$ về dạng $x = g(x)$, với $g(x)$ là một hàm số thỏa mãn tính chất: tồn tại số $q \in (0, 1)$ sao cho

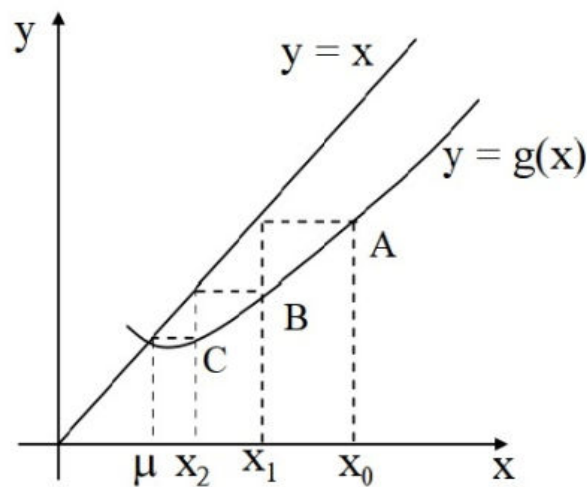
$$|g'(x)| \leq q \quad \forall x \in [a, b].$$

Bước 2: Chọn xấp xỉ ban đầu bất kỳ $x_0 \in [a, b]$.

Bước 3: Xây dựng dãy nghiệm xấp xỉ theo công thức

$$x_{n+1} = g(x_n) \quad n = 0, 1, 2, \dots$$

Ý tưởng hình học



Hình a

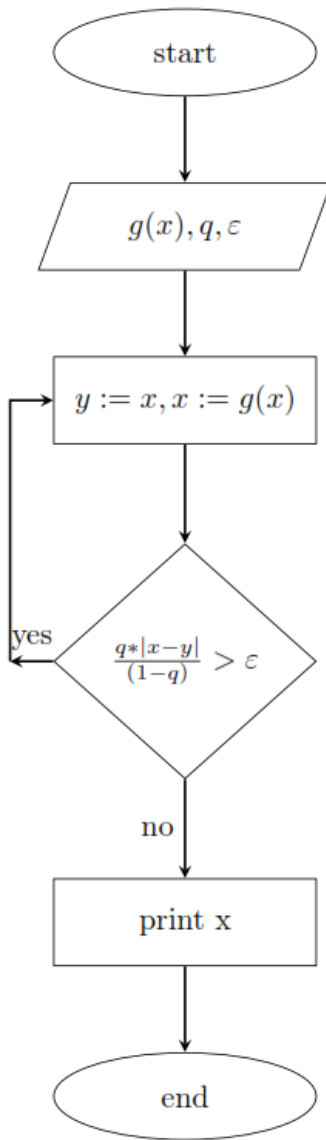
Sai số

Ta có công thức đánh giá sai số của nghiệm gần đúng x_n lấy ra từ dãy ở bước 3.

$$|x_n - x^*| \leq \frac{q}{1 - q} |x_n - x_{n-1}|$$

$$|x_n - x^*| \leq \frac{q^n}{1-q} |x_1 - x_0|$$

Sơ đồ khối:



Ví dụ:

Tìm nghiệm của phương trình $x^3 - x - 1 = 0$ sử dụng phương pháp lặp đơn để đạt được sai số $\varepsilon = 0.5e - 5$.

Gợi ý viết **hàm m-file (không dùng Run để chạy được Code example, phải viết code vào m-file riêng):**

```

function x1 = LapDon(fun, x0, a, b, eps)
%fun là hàm lặp dạng symbolic
%x0 là xấp xỉ ban đầu
%(a,b) là khoảng phân ly nghiệm
%eps là sai số mong muốn
    dfun = diff(fun);
% tìm hệ số co q
  
```

```

        h = 0.1; n = ceil((b-a)/0.1);
        t = linspace(a, b, n+1);
        y = subs(dfun, t);
        q = max(abs(y));
        q = double(q);
    saiSo = eps+1;
    while saiSo > eps
        x1 = double(subs(fun,x0));
        saiSo = q/(1-q)*abs(x1-x0);
        x0 = x1;
    end
end

```

Ví dụ gọi hàm LapDon cho phương trình $x^3 - x - 1 = 0$

```

syms x
% khai báo hàm lặp
fun = (1+x)^(1/3)
% gán các giá trị
a = 1, b = 1.5, x0 = 1, eps = 1.5,
r = LapDon(fun, x0, a, b, eps)

```

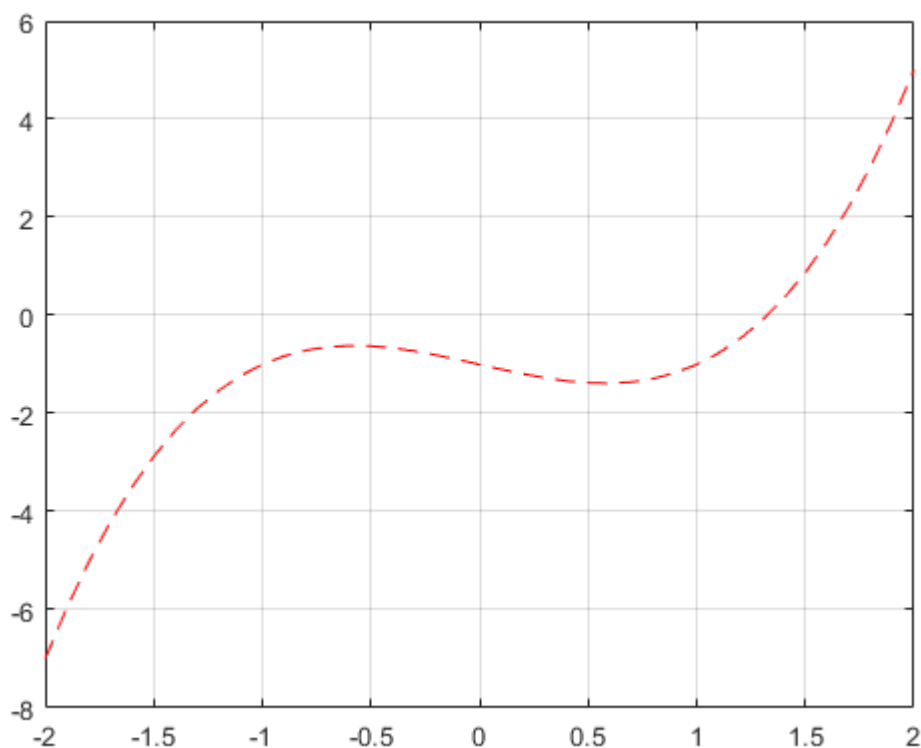
Giải ví dụ trên sử dụng thủ tục MATLAB

Bước 1: Tìm khoảng phân ly nghiệm

```

x = -2:.1:2;
y = x.^3-x-1;
plot(x,y,'-r')
grid on

```



Chứng minh $(1, 1.5)$ là khoảng phân ly nghiệm. Đặt $f(x) = x^3 - x - 1$. Ta có $f'(x) = 3x^2 - 1 > 2 \quad \forall x \in [1, 1.5]$. Suy ra hàm số $f(x)$ đồng biến trên $[1, 1.5]$. Hơn nữa, $f(1)f(1.5) = -0.8750 < 0$. Do đó, khoảng $(1, 1.5)$ là khoảng phân ly nghiệm của phương trình đã cho.

```
syms x
f = x^3-x-1
```

```
f = x^3 - x - 1
```

```
double(subs(f,1)*subs(f,1.5))
```

```
ans =
-0.8750000000000000
```

Bước 2: Biến đổi tương đương $x^3 - x - 1 \Leftrightarrow x = (x + 1)^{1/3}$. Chọn $g(x) = (x + 1)^{1/3}$.

Ta có $g'(x) = \frac{1}{3(x+1)^{2/3}}$. Dễ thấy $|g'(x)| \leq g'(1) = 0.2100 = q < 1 \quad \forall x \in [1, 1.5]$. Do đó phương pháp lặp đơn hội tụ.

```
syms x
g = (x+1)^(1/3)
```

```
g = (x + 1)^(1/3)
```

```
dg = diff(g)
```

```
dg =
    1
    3 (x + 1)^(2/3)
```

```
double(subs(dg,1))
```

```
ans =
    0.209986841649146
```

Bước 3: Chọn xấp xỉ $x_0 = 1$.

Bước 4: Xây dựng dãy nghiệm xấp xỉ theo công thức

$$x_{n+1} = (x_n + 1)^{1/3}, n = 0, 1, 2, \dots$$

```
x0 = 1
```

```
x0 =
    1
```

```
epsilon = 0.5e-5;
saiSo = 1;
while saiSo > epsilon
    x1 = (x0+1)^(1/3)
    saiSo = 0.21/(1-0.21)*abs(x1-x0);
    x0=x1;
end
```

```
x1 =
    1.259921049894873
x1 =
    1.312293836683289
x1 =
    1.322353819138825
x1 =
    1.324268744551578
x1 =
    1.324632625250920
x1 =
    1.324701748510359
x1 =
    1.324714878440950
```

```
format long
x1
```

```
x1 =
    1.324714878440950
```

Vậy nghiệm gần đúng cần tìm là $x = 1.324714878440950$.

Bài tập

Bài tập 1.

Viết chương trình MATLAB giải gần đúng phương trình $f(x) = 0$ tìm nghiệm trong khoảng (a, b) sử dụng phương pháp lặp đơn với n lần lặp. Kiểm tra chương trình với phương trình $x^3 + 3x^2 - 1 = 0$ và khoảng cách li $(-3, -2.5)$ và $n = 7$.

Tuần 10: Phương pháp lặp đơn giải hệ phương trình $Ax = b$.

Nội dung thực hành

1. Viết chương trình MATLAB cho phương pháp lặp đơn giải hệ phương trình

Mọi đóng góp xin liên hệ: tam.doduc@hust.edu.vn

Tóm tắt lý thuyết

Chuẩn của vector

Ví dụ:

```
x = [0.1 -0.3 0.2] %véc tơ hàng
```

```
x = 1×3  
    0.1000    -0.3000     0.2000
```

$$\|x\|_{\infty} = |x_1| + |x_2| + |x_3|$$

$$\|x\|_1 = \max\{|x_1|, |x_2|, |x_3|\}$$

```
normInf x = abs(0.1)+abs(-0.3) +abs(0.2)
```

```
normInf x = 0.6000
```

```
norm1 x = max(abs(x))
```

```
norm1 x = 0.3000
```

```
y = [0.1 -0.3 0.2]' %véc tơ cột
```

```
y = 3×1  
    0.1000  
   -0.3000  
    0.2000
```

$$\|y\|_{\infty} = \max\{|y_1|, |y_2|, |y_3|\} = \max\{|0.1|, |-0.3|, |0.2|\} = 0.3$$

$$\|y\|_1 = |y_1| + |y_2| + |y_3| = 0.1 + 0.3 + 0.2 = 0.6$$

Định nghĩa chuẩn của véc tơ cột:

$$\|x\|_1 = \sum_{i=1}^n |x_i|$$

$$\|x\|_{\infty} = \max_{i=1,2,3,\dots,n} |x_i|$$

Định nghĩa chuẩn của véc tơ hàng:

$$\|x\|_{\infty} = \sum_{i=1}^n |x_i|$$

$$\|x\|_1 = \max_{i=1,2,3,\dots,n} |x_i|$$

Chuẩn của ma trận

Ví dụ:

```
A = randi([-10,10],3,4)
```

```
A = 3x4
     7     9    -5    10
     9     3     1    -7
    -8    -8    10    10
```

```
sum(abs(A),2)
```

```
ans = 3x1
     31
     20
     36
```

```
normInfA = max(sum(abs(A),2))% chuẩn hàng của ma trận A
```

```
normInfA = 36
```

```
sum(abs(A),1)
```

```
ans = 1x4
     24     20     16     27
```

```
norm1A = max(sum(abs(A),1)) % chuẩn cột của ma trận A
```

```
norm1A = 27
```

$$\|A\|_{\infty} = \max_{i=1,m} \left\{ \sum_{j=1}^n |a_{ij}| \right\}$$

$$\|A\|_1 = \max_{j=1,n} \left\{ \sum_{i=1}^m |a_{ij}| \right\}$$

Nhận xét: Độ lớn phần tử bất kỳ của ma trận A không thể vượt quá chuẩn của nó (chuẩn hàng hoặc chuẩn cột).

Ta nói 2 ma trận cũng kích cỡ A và B gần nhau nếu như $\|A - B\|$ là nhỏ.

Sự hội tụ của dãy véc tơ

Ví dụ:

```
for n = 10000:10005
    [1 1/n sin(n)/n]; % dãy véc tơ hội tụ đến [1 0 0]
end
```

Điều này có nghĩa là $\|[1 \ 1/n \ \sin(n)/n] - [1 \ 0 \ 0]\|$ tiến về 0 khi n tiến ra vô cùng.

```
syms k
```

$$v = [1 \ 1/k \ \sin(k)/k] - [1 \ 0 \ 0]$$

$$v = \begin{pmatrix} 0 & \frac{1}{k} & \frac{\sin(k)}{k} \end{pmatrix}$$

$$\text{chuanVectov} = 1/k + \text{abs}(\sin(k))/k$$

$$\text{chuanVectov} =$$

$$\frac{|\sin(k)|}{k} + \frac{1}{k}$$

$$\text{double}(\text{limit}(\text{chuanVectov}, k, \text{inf}))$$

$$\text{ans} = 0$$

Dãy vector $x^{(k)} = (x_1^{(k)}, x_2^{(k)}, \dots, x_m^{(k)})^T$ gọi là hội tụ đến vector $x = (x_1, x_2, \dots, x_m)^T$ và ký hiệu là $\lim_{k \rightarrow \infty} x^{(k)} = x$, nếu

$$\lim_{k \rightarrow \infty} x_i^{(k)} = x_i, \quad \forall i = \overline{1, m} \text{ (hội tụ theo phần tử)}$$

hoặc

$$\|x^{(k)} - x\|_p \rightarrow 0 \text{ khi } k \rightarrow \infty, p = \infty, 1, 2 \text{ (hội tụ theo chuẩn)}.$$

Phương pháp lặp đơn giải gần đúng hệ phương trình tuyến tính

Bài toán: Cho hệ phương trình $X = BX + d$ trong đó B là ma trận vuông cấp $n \times n$, X, d là hai véc tơ cột kích thước $n \times 1$. Tìm nghiệm gần đúng của hệ đã cho?

Ví dụ: xét hệ $X = \alpha X + \beta$

$$\alpha = \begin{pmatrix} 0 & -0,06 & 0,02 \\ -0,03 & 0 & 0,05 \\ -0,01 & 0,02 & 0 \end{pmatrix}; \beta = \begin{pmatrix} 2 \\ 3 \\ 5 \end{pmatrix}$$

$$\begin{cases} x_1 = 0x_1 - 0.06x_2 + 0.02x_3 + 2 \\ x_2 = -0.03x_1 + 0x_2 + 0.05x_3 + 3 \\ x_3 = -0.01x_1 + 0.02x_2 + 0x_3 + 5 \end{cases}$$

Điều kiện thực hiện phương pháp lặp đơn: $\|B\| = q < 1$.

Nội dung phương pháp:

1. Kiểm tra điều kiện hội tụ
2. Chọn véc tơ xấp xỉ ban đầu bất kỳ X_0 .
3. Xây dựng dãy véc tơ theo công thức $X_{n+1} = BX_n + d$.
4. Chọn X_n làm nghiệm gần đúng và đánh giá theo sai số

$$\|X_n - X^*\| \leq \frac{q^n}{1-q} \|X_1 - X_0\|$$

$$\|X_n - X^*\| \leq \frac{q}{1-q} \|X_n - X_{n-1}\|$$

Ví dụ: Xét hệ ở ví dụ phía trên

$$\begin{cases} x_1 = 0x_1 - 0.06x_2 + 0.02x_3 + 2 \\ x_2 = -0.03x_1 + 0x_2 + 0.05x_3 + 3 \\ x_3 = -0.01x_1 + 0.02x_2 + 0x_3 + 5 \end{cases}$$

Bước 1 Viết lại hệ trên dưới dạng $x = \alpha x + \beta$ trong đó

$$\alpha = \begin{pmatrix} 0 & -0.06 & 0.02 \\ -0.03 & 0 & 0.05 \\ -0.01 & 0.02 & 0 \end{pmatrix}; \beta = \begin{pmatrix} 2 \\ 3 \\ 5 \end{pmatrix}$$

Ta có: $\|\alpha\|_{\infty} = \max(0.08; 0.08; 0.03) = 0.08 < 1$ Do đó dãy vector $x^{(k)} = \alpha x^{(k-1)} + \beta$ hội tụ đến nghiệm đúng x^* của hệ.

```
alpha = [0 -0.06 0.02;-0.03 0 0.05;-0.01 0.02 0]
```

```
alpha = 3x3
      0    -0.0600    0.0200
    -0.0300     0    0.0500
    -0.0100    0.0200     0
```

```
q = max(sum(abs(alpha),2))
```

```
q = 0.0800
```

```
beta = [2;3;5]
```

```
beta = 3x1
      2
      3
      5
```

Bước 2. Chọn xấp xỉ đầu $x^{(0)} = (0, 0, 0)^T$.

```
X0 = [0;0;0]
```

```
X0 = 3x1
      0
      0
      0
```

Bước 3. Tính các nghiệm xấp xỉ theo công thức

$$x^{(k)} = \alpha x^{(k-1)} + \beta$$

```
for n = 1:10
    X1 = alpha*X0+beta
```

```

    saiSo = q/(1-q)*max(abs(X1-X0))
    X0=X1;
end

```

```

X1 = 3×1
    2
    3
    5
saiSo = 0.4348
X1 = 3×1
    1.9200
    3.1900
    5.0400
saiSo = 0.0165
X1 = 3×1
    1.9094
    3.1944
    5.0446
saiSo = 9.2174e-04
X1 = 3×1
    1.9092
    3.1949
    5.0448
saiSo = 4.7652e-05
X1 = 3×1
    1.9092
    3.1950
    5.0448
saiSo = 2.5217e-06
X1 = 3×1
    1.9092
    3.1950
    5.0448
saiSo = 1.3078e-07
X1 = 3×1
    1.9092
    3.1950
    5.0448
saiSo = 6.8257e-09
X1 = 3×1
    1.9092
    3.1950
    5.0448
saiSo = 3.6329e-10
X1 = 3×1
    1.9092
    3.1950
    5.0448
saiSo = 1.8746e-11
X1 = 3×1
    1.9092
    3.1950
    5.0448
saiSo = 1.0205e-12

```

Chọn $X^{(10)} = (1.9092, 3.1950, 5.0448)^T$ là nghiệm gần đúng với sai số $1.0205e-12$.

Gợi ý code cho bài toán: Cho hệ phương trình dạng $x = \alpha x + \beta$. Cho xấp xỉ ban đầu $x^{(0)}$, sử dụng phương pháp lặp đơn tìm nghiệm gần đúng sau k lần lặp của hệ và đánh giá sai số?

```

function [X1, saiSo] = lapDon(alpha, beta, X0, k)
    X1=alpha*X0+beta;

```

```

for i = 1:(k-1)
    X0=X1;
    X1=alpha*X0+beta;
end
q = norm(alpha, inf);
saiSo = q/(1-q)*norm(X1-X0,inf);
end

```

Trong cửa sổ **Command Window** gọi hàm như sau:

```

>> alpha = [0 -0.06 0.02;-0.03 0 0.05;-0.01 0.02 0];
>> beta = [2;3;5];
>> X0 = [0;0;0];
>> k = 10;
>> [X10, saiSo] = lapDon(alpha, beta, X0, k)

```

Bài tập

Bài tập 1: Viết chương trình MATLAB nhập vào hệ phương trình $x = \alpha x + \beta$ và xấp xỉ ban đầu $x^{(0)} = \beta$. Tìm nghiệm gần đúng của hệ đạt sai số $\varepsilon = 1e - 5$? Kiểm tra chương trình với hệ phương trình sau

$$\begin{cases} 15.60x_1 - 2.73x_2 + 1.89x_3 = 6.75 \\ 2.50x_1 - 16.50x_2 + 7.40x_3 = 2.86 \\ 3.00x_1 + 11.56x_2 + 27.90x_3 = 9.85 \end{cases}$$

Đa thức nội suy Lagrange

Mọi đóng góp xin liên hệ: tam.doduc@hust.edu.vn

Bài toán

Cho bảng giá trị dạng

$$\begin{array}{c|cccc} x & x_0 & x_1 & \cdots & x_n \\ \hline y & y_0 & y_1 & \cdots & y_n \end{array}$$

của hàm số $y = f(x)$ (không nhất thiết biết biểu thức của $f(x)$) xác định trên đoạn $[a, b]$. Tìm đa thức $P(x)$ thỏa mãn

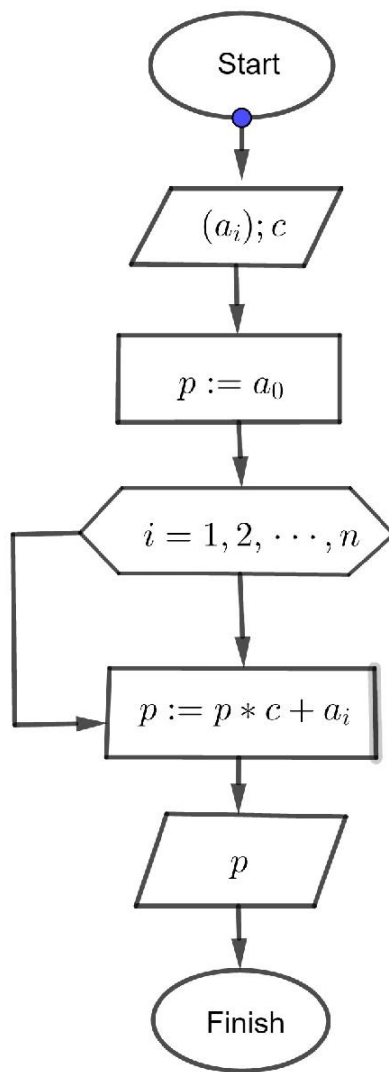
$$\begin{cases} \deg P(x) \leq n & (\text{số điểm } -1), \\ P(x_i) = y_i, & \text{với mọi } i = \overline{0, n}. \end{cases}$$

Lược đồ Hoocner

Cho đa thức bậc n có dạng tổng quát :

$$p(x) = a_0x_n + a_1x_{n-1} + \cdots + a_{n-1}x + a_n \quad (a_0 \neq 0)$$

Tính giá trị đa thức $p(x)$ khi $x = c$ (c : giá trị cho trước) và tìm thương và dư của phép chia $p(x)$ cho đơn thức $(x - c)$.



Gợi ý code MATLAB chia đa thức $p(x)$ cho đơn thức $(x - c)$.

```

function [q, r] = hoocnerChia(x,c)
n=length(x);
q=zeros(1,n);
q(1)=x(1);
for i = 2:n
    q(i)=c*q(i-1)+x(i);
end
r = q(end);
q(end)=[];
end

```

Gợi ý code MATLAB nhân đa thức $p(x)$ với đơn thức $(x - c)$.

```

function q = hoocnerNhan(p, c)
n = length(p);
p = p(:)';
p = [p, 0];
q = zeros(1, n+1);
q(1) = p(1);
for i = 1:n
    q(i+1) = p(i+1) - c*p(i);
end

```

end
end

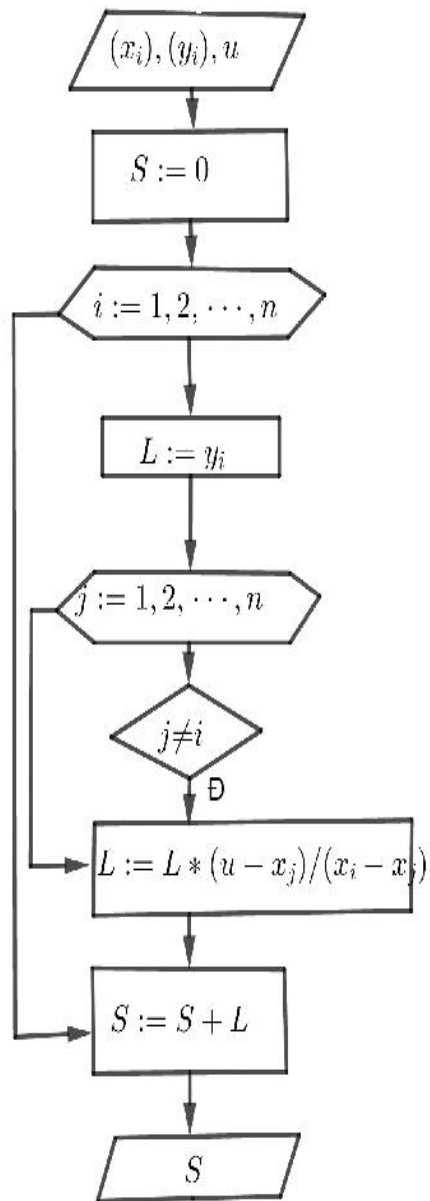
Đa thức nội suy Lagrange

Đa thức cơ sở Lagrange bậc n

$$L_j(x) = \frac{(x - x_0)(x - x_1) \cdots (x - x_{j-1})(x - x_{j+1})(x - x_n)}{(x_j - x_0)(x_j - x_1) \cdots (x_j - x_{j-1})(x_j - x_{j+1})(x_j - x_n)}, \quad \forall j = \overline{0, n}.$$

Đa thức nội suy Lagrange

$$P_n(x) = \sum_{j=0}^n L_j(x)y_j.$$



Gợi ý code MATLAB đưa ra hệ số dạng chính tắc của đa thức nội suy Lagrange

```

function L = Lagrange(x, y)
% x là vecto chứa các mốc nội suy
% y là vecto chứa giá trị tương ứng tại các mốc nội suy
n = length(x);
L = zeros(1, n);
for j = 1:n
    S = 1;
    for i = 1:n
        if i ~= j
            S = conv(S, [1 -x(i)]./(x(j)-x(i)));
            length(S)
        end
    end
    L = L + y(j)*S;
end
end
  
```


Tuần 11: Đa thức nội suy Lagrange

Nội dung thực hành

1. Viết chương trình MATLAB tìm đa thức nội suy Lagrange

Mọi đóng góp xin liên hệ: tam.doduc@hust.edu.vn

Tóm tắt lý thuyết

Bài toán

Cho bảng giá trị dạng

$$\begin{array}{c|cccc} x & x_0 & x_1 & \cdots & x_n \\ \hline y & y_0 & y_1 & \cdots & y_n \end{array}$$

của hàm số $y = f(x)$ (không nhất thiết biết biểu thức của $f(x)$) xác định trên đoạn $[a, b]$. Tìm đa thức $P(x)$ thỏa mãn

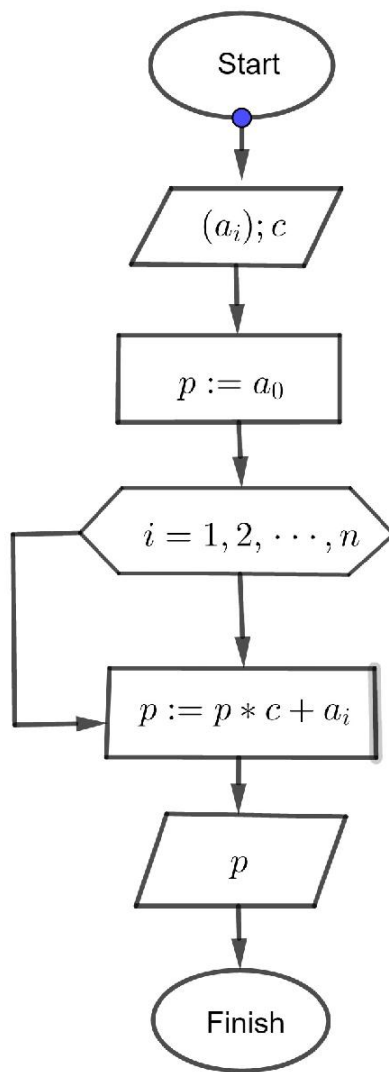
$$\begin{cases} \deg P(x) \leq n & (\text{số điểm } -1), \\ P(x_i) = y_i, & \text{với mọi } i = \overline{0, n}. \end{cases}$$

Lược đồ Hoocner

Cho đa thức bậc n có dạng tổng quát :

$$p(x) = a_0x_n + a_1x_{n-1} + \cdots + a_{n-1}x + a_n \quad (a_0 \neq 0)$$

Tính giá trị đa thức $p(x)$ khi $x = c$ (c : giá trị cho trước) và tìm thương và dư của phép chia $p(x)$ cho đơn thức $(x - c)$.



Gợi ý code MATLAB chia đa thức $p(x)$ cho đơn thức $(x - c)$.

```

function [q, r] = hoocnerChia(x,c)
n=length(x);
q=zeros(1,n);
q(1)=x(1);
for i = 2:n
    q(i)=c*q(i-1)+x(i);
end
r = q(end);
q(end)=[];
end
  
```

Gợi ý code MATLAB nhân đa thức $p(x)$ với đơn thức $(x - c)$.

```

function q = hoocnerNhan(p, c)
n = length(p);
p = p(:)';
p = [p, 0];
q = zeros(1, n+1);
q(1) = p(1);
for i = 1:n
    q(i+1) = p(i+1) - c*p(i);
end
  
```

end
end

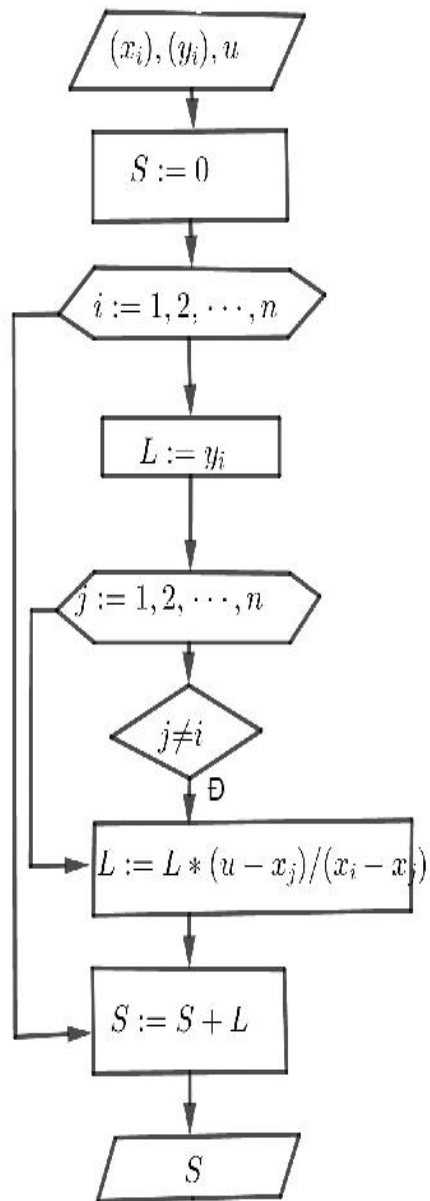
Đa thức nội suy Lagrange

Đa thức cơ sở Lagrange bậc n

$$L_j(x) = \frac{(x - x_0)(x - x_1) \cdots (x - x_{j-1})(x - x_{j+1})(x - x_n)}{(x_j - x_0)(x_j - x_1) \cdots (x_j - x_{j-1})(x_j - x_{j+1})(x_j - x_n)}, \quad \forall j = \overline{0, n}.$$

Đa thức nội suy Lagrange

$$P_n(x) = \sum_{j=0}^n L_j(x)y_j.$$



Gợi ý code MATLAB đưa ra hệ số dạng chính tắc của đa thức nội suy Lagrange

```

function L = Lagrange(x, y)
% x là vecto chứa các mốc nội suy
% y là vecto chứa giá trị tương ứng tại các mốc nội suy
n = length(x);
L = zeros(1, n);
for j = 1:n
    S = 1;
    for i = 1:n
        if i ~= j
            S = conv(S, [1 -x(i)]./(x(j)-x(i)));
            length(S)
        end
    end
    L = L + y(j)*S;
end
end

```

Bài tập

Bài tập 1. Viết thuật toán đưa ra giá trị gần đúng của hàm số $f(x)$ tại điểm x_0 khi biết bảng giá trị của $f(x)$ sử dụng đa thức nội suy Lagrange? Kiểm tra chương trình với $x_0 = 1.5$ và bộ dữ liệu $x = 0 : 3$ và $y = [-5 \quad -6 \quad -1 \quad 16]$.

Tuần 12: Đa thức nội suy Lagrange

Nội dung thực hành

1. Viết chương trình MATLAB tìm đa thức nội suy Newton

Mọi đóng góp xin liên hệ: tam.doduc@hust.edu.vn

Tóm tắt lý thuyết

Bài toán

Cho bảng giá trị dạng

$$\begin{array}{c|cccc} x & x_0 & x_1 & \cdots & x_n \\ \hline y & y_0 & y_1 & \cdots & y_n \end{array}$$

của hàm số $y = f(x)$ (không nhất thiết biết biểu thức của $f(x)$) xác định trên đoạn $[a, b]$. Tìm đa thức $P(x)$ thỏa mãn

$$\begin{cases} \deg P(x) \leq n & (\text{số điểm } -1), \\ P(x_i) = y_i, & \text{với mọi } i = \overline{0, n}. \end{cases}$$

Đa thức nội suy Newton mốc bất kỳ

Tỷ sai phân của hàm số tại các mốc nội suy được định nghĩa quy nạp như sau:

- Tỷ sai phân cấp 1

$$f\{x_k, x_{k+1}\} = \frac{y_{k+1} - y_k}{x_{k+1} - x_k}$$

- Tỷ sai phân cấp 2

$$f\{x_k, x_{k+1}, x_{k+2}\} = \frac{f\{x_{k+1}, x_{k+2}\} - f\{x_k, x_{k+1}\}}{x_{k+2} - x_k}$$

- Tỷ sai phân cấp s

$$f\{x_k, \dots, x_{k+s}\} = \frac{f\{x_{k+1}, \dots, x_{k+s}\} - f\{x_k, x_{k+1}, \dots, x_{k+s-1}\}}{x_{k+s} - x_k}$$

Công thức nội suy Newton mốc bất kỳ

$$\begin{aligned}
P(x) = & y_0 + f[x_0, x_1](x - x_0) + f[x_0, x_1, x_2](x - x_0)(x - x_1) \\
& + f[x_0, \dots, x_3](x - x_0)(x - x_1)(x - x_2) \\
& + \dots \\
& + f[x_0, \dots, x_n](x - x_0) \dots (x - x_{n-1}).
\end{aligned}$$

Gợi ý code MATLAB tìm đa thức nội suy Newton tiến mốc bất kỳ

```

function [p, y, val] = NewtonTienMBK(x, y, u)
n = length(x);
%tinh ty sai phan cac cap chua moc y0
for j = 2:n
    for i = n:-1:j
        y(i) = (y(i)-y(i-1))/(x(i)-x(i-j+1));
    end
end
p = y(end);
% dua da thuc ve dang chinh tac
for j = n-1:-1:1
    p = [p y(j)] - [0 p*x(j)];
end
%tinh gia tri da thuc su dung so do hoocne
val = p(1);
for j = 1:n-1
    val = u.*val + p(j+1);
end

```

Đa thức nội suy Newton mốc cách đều

Sai phân của hàm số tại các mốc nội suy được định nghĩa quy nạp như sau:

- Sai phân tiến cấp 1 từ x_k và sai phân lùi cấp 1 từ x_{k+1}

$$\Delta y_k = y_{k+1} - y_k = \nabla y_{k+1}$$

- Sai phân tiến cấp 2 từ x_k và sai phân lùi cấp 2 từ x_{k+2}

$$\Delta^2 y_k = \Delta y_{k+1} - \Delta y_k = \nabla y_{k+2} - \nabla y_{k+1} = \nabla^2 y_{k+2}$$

- Sai phân tiến cấp s từ x_k và sai phân lùi cấp s từ x_{k+s}

$$\Delta^s y_k = \Delta^{s-1} y_{k+1} - \Delta^{s-1} y_k = \nabla^{s-1} y_{k+s} - \nabla^{s-1} y_{k+s-1} = \nabla^s y_{k+s}$$

Công thức nội suy Newton tiến mốc cách đều

$$P(x_0 + th) = y_0 + \Delta y_0 t + \frac{\Delta^2 y_0}{2!} t(t-1) + \frac{\Delta^3 y_0}{3!} t(t-1)(t-2) + \dots + \frac{\Delta^n y_0}{n!} t(t-1) \dots (t-n+1).$$

Gợi ý code MATLAB tìm đa thức nội suy Newton tiến mốc cách đều

```
clc; clear all; format shorte
x=[1.0000 1.1000 1.2000 1.3000 1.4000 1.5000 1.6000];
%y =[0.8415 0.9803 1.1184 1.2526 1.3796 1.4962 1.5993];
y =[0.5403 0.4990 0.4348 0.3477 0.2380 0.1061 -0.0467];
n = length(x);
A = diag(-1*ones(1,n)) + diag(ones(1,n-1),1);
z = y;
Heso = zeros(1,n);
Heso(1) = z(1);
for i = 1:n
    z = A*z(:);
    if i<n
        Heso(i+1) = z(1);
    end
end
disp('Sai phan cac cap tai moc y1 la:');
disp(Heso);
disp('He so cua da thuc noi suy Newton tien MCD')
for i = 1:length(Heso)
    p1(i) = Heso(i)/factorial(i-1);
end
p1
p=Heso(end)/factorial(n-1);

for k = n-1:-1:1
    p = [p Heso(k)/factorial(k-1)]-[0 p*(k-1)];
end
disp('He so cua da thuc noi suy Newton tien MCD trong dang chinh tac la:');
disp(p);
L= poly2sym(p)
disp('Gia tri gan dung cua ham so tai x = 1.05')
double(subs(L,0.5))
```

Bài tập

Bài tập 1.

Viết thuật toán và chương trình MATLAB đưa ra đa thức nội suy Newton lùi mốc bất kỳ từ bảng giá trị cho trước. Kiểm tra chương trình với bộ dữ liệu

$x = [-4.2500 \quad -2.0000 \quad 1.0000 \quad -3.0000 \quad -1.2500 \quad -3.7500]$ và

$y = [-1.7956 \quad -1.9869 \quad 1.3105 \quad -0.0201 \quad 2.8560 \quad -0.3981]$.

Bài tập 2.

Viết thuật toán và chương trình MATLAB đưa ra đa thức nội suy Newton lùi mốc cách đều từ bảng giá trị cho trước. Kiểm tra chương trình với bộ dữ liệu $x = 1 : 0.5 : 4$ và $y = [0.8415 \ 2.2444 \ 3.6372 \ 3.7405 \ 1.2701 \ -4.2971 \ -12.1088]$.

Bài toán

Cho bảng giá trị dạng

$$\begin{array}{c|cccc} x & x_0 & x_1 & \cdots & x_n \\ \hline y & y_0 & y_1 & \cdots & y_n \end{array}$$

của hàm số $y = f(x)$ (không nhất thiết biết biểu thức của $f(x)$) xác định trên đoạn $[a, b]$. Tìm đa thức $P(x)$ thỏa mãn

$$\begin{cases} \deg P(x) \leq n & (\text{số điểm } -1), \\ P(x_i) = y_i, & \text{với mọi } i = \overline{0, n}. \end{cases}$$

Đa thức nội suy Newton mốc bất kỳ

Tỷ sai phân của hàm số tại các mốc nội suy được định nghĩa quy nạp như sau:

- Tỷ sai phân cấp 1

$$f\{x_k, x_{k+1}\} = \frac{y_{k+1} - y_k}{x_{k+1} - x_k}$$

- Tỷ sai phân cấp 2

$$f\{x_k, x_{k+1}, x_{k+2}\} = \frac{f\{x_{k+1}, x_{k+2}\} - f\{x_k, x_{k+1}\}}{x_{k+2} - x_k}$$

- Tỷ sai phân cấp s

$$f\{x_k, \dots, x_{k+s}\} = \frac{f\{x_{k+1}, \dots, x_{k+s}\} - f\{x_k, x_{k+1}, \dots, x_{k+s-1}\}}{x_{k+s} - x_k}$$

Công thức nội suy Newton mốc bất kỳ

$$\begin{aligned} P(x) = & y_0 + f[x_0, x_1](x - x_0) + f[x_0, x_1, x_2](x - x_0)(x - x_1) \\ & + f[x_0, \dots, x_3](x - x_0)(x - x_1)(x - x_2) \\ & + \dots \\ & + f[x_0, \dots, x_n](x - x_0) \dots (x - x_{n-1}). \end{aligned}$$

Gợi ý code MATLAB tìm đa thức nội suy Newton tiến mốc bất kỳ

```
function [p, y, val] = NewtonTienMBK(x, y, u)
n = length(x);
%tính tỷ sai phân các cấp chưa mốc y0
for j = 2:n
    for i = n:-1:j
```

```

        y(i) = (y(i)-y(i-1))/(x(i)-x(i-j+1));
    end
end
p = y(end);
% đưa đa thức về dạng chính tắc
for j = n-1:-1:1
    p = [p y(j)] - [0 p*x(j)];
end
% tính giá trị đa thức sử dụng sơ đồ hoocne
val = p(1);
for j = 1:n-1
    val = u.*val + p(j+1);
end

```

Đa thức nội suy Newton mốc cách đều

Sai phân của hàm số tại các mốc nội suy được định nghĩa quy nạp như sau:

- Sai phân tiến cấp 1 từ x_k và sai phân lùi cấp 1 từ x_{k+1}

$$\Delta y_k = y_{k+1} - y_k = \nabla y_{k+1}$$

- Sai phân tiến cấp 2 từ x_k và sai phân lùi cấp 2 từ x_{k+2}

$$\Delta^2 y_k = \Delta y_{k+1} - \Delta y_k = \nabla y_{k+2} - \nabla y_{k+1} = \nabla^2 y_{k+2}$$

- Sai phân tiến cấp s từ x_k và sai phân lùi cấp s từ x_{k+s}

$$\Delta^s y_k = \Delta^{s-1} y_{k+1} - \Delta^{s-1} y_k = \nabla^{s-1} y_{k+s} - \nabla^{s-1} y_{k+s-1} = \nabla^s y_{k+s}$$

Công thức nội suy Newton tiến mốc cách đều

$$\begin{aligned}
 P(x_0 + th) = & y_0 + \Delta y_0 t + \frac{\Delta^2 y_0}{2!} t(t-1) + \frac{\Delta^3 y_0}{3!} t(t-1)(t-2) \\
 & + \dots + \frac{\Delta^n y_0}{n!} t(t-1) \dots (t-n+1).
 \end{aligned}$$

Gợi ý code MATLAB tìm đa thức nội suy Newton tiến mốc cách đều

```

clc; clear all; format shorte
x=[1.0000 1.1000 1.2000 1.3000 1.4000 1.5000 1.6000];
%y=[0.8415 0.9803 1.1184 1.2526 1.3796 1.4962 1.5993];
y=[0.5403 0.4990 0.4348 0.3477 0.2380 0.1061 -0.0467];
n = length(x);
A = diag(-1*ones(1,n)) + diag(ones(1,n-1),1);
z = y;
Heso = zeros(1,n);
Heso(1) = z(1);
for i = 1:n
    z = A*z(:);
    if i<n

```

```

        Heso(i+1) = z(1);
    end
end
disp('Sai phan cac cap tai moc y1 la:');
disp(Heso);
disp('He so cua da thuc noi suy Newton tien MCD')
for i = 1:length(Heso)
    p1(i) = Heso(i)/factorial(i-1);
end
p1
p=Heso(end)/factorial(n-1);

for k = n-1:-1:1
    p = [p Heso(k)/factorial(k-1)]-[0 p*(k-1)];
end
disp('He so cua da thuc noi suy Newton tien MCD trong dang chinh tac la:');
disp(p);
L= poly2sym(p)
disp('Gia tri gan dung cua ham so tai x = 1.05')
double(subs(L,0.5))

```

Tuần 13: Bình phương tối thiểu

Nội dung thực hành

1. Viết chương trình MATLAB cho phương pháp bình phương tối thiểu

Mọi đóng góp xin liên hệ: tam.doduc@hust.edu.vn

Tóm tắt lý thuyết

Nội dung phương pháp

Cho các hàm số

$$\varphi_k(x), \quad k = \overline{0, m}$$

là các hàm số 1 biến số phụ thuộc vào x . Hàm số dạng

$$\varphi(x) = \sum_{k=0}^m a_k \varphi_k(x)$$

gọi là đa thức suy rộng của hệ hàm trên.

Ví dụ:

- $\varphi_k(x) = x^k, k = \overline{0, m}$ gọi là hệ đại số;
- $1, \cos x, \sin x, \cos 2x, \sin 2x, \dots$ gọi là hệ lượng giác;
- $1, e^{\alpha_1 x}, e^{\alpha_2 x}, e^{\alpha_3 x}, \dots$ trong đó $\{\alpha_i\}$ – gọi là hệ hàm số mũ.

Xấp xỉ hàm số $f(x)$ bởi hàm số $\varphi(x)$ bằng phương pháp bình phương tối thiểu là đi tìm các hệ số $a_0, a_1, \dots, a_m$ sao cho đại lượng

$$\sigma_m = \Phi(a_0, a_1, \dots, a_m) = \sum_{i=0}^n \left(y_i - \sum_{k=0}^m a_k \varphi_k(x_i) \right)^2$$

đạt min trên phân hoạch $\Delta = \{a = x_1 < x_2 < \dots < x_n = b\}$.

Khi đó $a_0, a_1, \dots, a_m$ là nghiệm của hệ $\frac{\partial \Phi}{\partial a_i} = 0, \quad \forall i = \overline{0, m}$. Do đó, để hàm số $\varphi(x)$ xấp xỉ tốt nhất hàm số $f(x)$

theo nghĩa bình phương tối thiểu thì các hệ số $a_0, a_1, \dots, a_m$ phải là nghiệm của hệ phương trình

$$(*) \left\{ \begin{array}{l} [\varphi_0, \varphi_0]a_0 + [\varphi_0, \varphi_1]a_1 + \dots + [\varphi_0, \varphi_m]a_m = [y, \varphi_0] \\ [\varphi_1, \varphi_0]a_0 + [\varphi_1, \varphi_1]a_1 + \dots + [\varphi_1, \varphi_m]a_m = [y, \varphi_1] \\ \dots \\ [\varphi_m, \varphi_0]a_0 + [\varphi_m, \varphi_1]a_1 + \dots + [\varphi_m, \varphi_m]a_m = [y, \varphi_m] \end{array} \right.$$

trong đó tích vô hướng được định nghĩa như sau

$$[\varphi_r, \varphi_k] := \sum_{i=1}^n \varphi_r(x_i) \varphi_k(x_i)$$

và vector

$$y = (y_1, y_2, \dots, y_m)^T.$$

$$\text{Đặt } Q = \begin{bmatrix} \varphi_0(x_0) & \varphi_0(x_1) & \cdots & \varphi_0(x_n) \\ \varphi_1(x_0) & \varphi_1(x_1) & \cdots & \varphi_1(x_n) \\ \vdots & \vdots & \vdots & \vdots \\ \varphi_m(x_0) & \varphi_m(x_1) & \cdots & \varphi_m(x_n) \end{bmatrix}.$$

Hệ (*) có thể viết lại dưới dạng ma trận như sau: $(QQ^T)A = QY \Leftrightarrow A = (QQ^T)^{-1}QY$

Gợi ý code MATLAB cho phương pháp bình phương tối thiểu:

```
function heSo = LeastSquare(x, y, Phi)
%[x; y] là bảng B
% Phi là vecto hàng chứa các hàm cơ sở
% Phi = [phi_0, phi_1, .., phi_m]
Q = double(subs(Phi(:), x)); % x phải là véc tơ hàng
y = y(:);
heSo = (Q*Q') \ (Q*y);
end
```

Bài tập

Bài tập 1:

Viết thuật toán và chương trình MATLAB tính gần đúng giá trị của hàm số $f(x)$ khi biết bảng giá trị của nó sử dụng phương pháp bình phương tối thiểu với dạng hàm thực nghiệm cho trước. Kiểm tra chương trình với hàm thực nghiệm dạng $y = ae^{bt}$ và bộ dữ liệu $t = [26.67 \ 93.33 \ 148.89 \ 315.56]$ và $y = [0.35 \ 0.085 \ 0.012 \ 0.0075]$.

Tuần 14: Tính gần đúng đạo hàm và tích phân

Nội dung thực hành

1. Viết chương trình MATLAB tính gần đúng tích phân đạo hàm.
2. Viết chương trình MATLAB tính gần đúng tích phân sử dụng công thức hình thang.
3. Viết chương trình MATLAB tính gần đúng tích phân sử dụng công thức Simpson.

Mọi đóng góp xin liên hệ: tam.doduc@hust.edu.vn

Tóm tắt lý thuyết

Tính gần đúng đạo hàm

- Công thức 2 điểm: với 2 điểm nội suy $(x_k, y_k), (x_{k+1}, y_{k+1})$, đạo hàm bậc nhất của hàm số $y = f(x)$ tại 2 mốc nội suy tương ứng có thể được tính theo công thức:

$$f'(x_k) \approx \frac{y_{k+1} - y_k}{h} \approx f'(x_{k+1}).$$

- Công thức 3 điểm: với 3 điểm nội suy $(x_k, y_k), (x_{k+1}, y_{k+1}), (x_{k+2}, y_{k+2})$, đạo hàm bậc nhất của hàm số $y = f(x)$ tại 3 mốc nội suy tương ứng có thể được tính theo công thức:

$$f'(x_k) \approx \frac{-3y_k + 4y_{k+1} + y_{k+2}}{2h} \quad (\text{Công thức cận trái})$$

$$f'(x_k) \approx \frac{-y_{k-1} + y_{k+1}}{2h} \quad (\text{Công thức trung tâm})$$

$$f'(x_k) \approx \frac{y_{k-2} - 4y_{k-1} + 3y_k}{2h} \quad (\text{Công thức cận phải})$$

Gợi ý code MATLAB: Cho bảng dữ liệu

$$\begin{array}{cccc} x & x_1 & x_2 & \cdots & x_n \\ y & y_1 & y_2 & \cdots & y_n \end{array}$$

Tính gần đúng đạo hàm của hàm số tại các mốc nội suy.

```
function Df = DaoHam(x,y,h)
Df = zeros(size(x));
% Tính gần đúng đạo hàm tại đầu bảng và cuối bảng
% sử dụng công thức hai điểm.
Df(1) = (y(2)-y(1))/(h);
Df(end) = (y(end)-y(end-1))/(h);
% Tính gần đúng đạo hàm tại các điểm còn lại sử dụng
% công thức trung tâm
for i = 2: length(Df)-1
    Df(i) = (y(i+1)-y(i-1))/(2*h);
end
end
```

Bài tập: Viết chương trình MATLAB tính gần đúng đạo hàm sử dụng các công thức cận trái và công thức cận phải.

Tính gần đúng tích phân

Chia $[a, b]$ thành n đoạn lấy tích phân bằng nhau

$$[a, b] = \cup_{i=0}^{n-1} [x_i, x_{i+1}], x_i = a + ih, i = \overline{0, n-1}.$$

công thức hình thang áp dụng trên toàn đoạn lấy tích phân $[a, b]$:

$$I = \int_a^b f(x)dx \approx I_h = \sum_{i=1}^n \frac{h}{2} (y_i + y_{i+1}) = \frac{h}{2} (y_1 + y_n + 2 \sum_{i=2}^{n-1} y_i)$$

Gợi ý code MATLAB: Cho bảng dữ liệu

$$\frac{x|x_1 \quad x_2 \quad \cdots \quad x_n}{y|y_1 \quad y_2 \quad \cdots \quad y_n}.$$

Tính gần đúng tích phân $\int_a^b f(x)dx$ sử dụng công thức hình thang.

```
function [Iht, ss] = hìnhThang(x, y, h)
% số mốc chia là số lẻ
% h = x(2)-x(1);
% bước lưới mịn
Iht = h/2*(y(1)+y(end)+2*sum(y(2:end-1)));
% bước lưới thưa
I2h = h*(y(1)+y(end)+2*sum(y(3:2:end-2)));
% đánh giá sai số theo 2 bước liên tiếp
ss = 1/3*abs(Iht - I2h);
end
```

Công thức Simpson áp dụng trên toàn đoạn lấy tích phân $[a, b]$:

$$I = \int_a^b f(x)dx \approx I_h = \sum_{i=1}^n \frac{h}{3} (y_{2i-1} + 4y_{2i} + y_{2i+1}) = \frac{h}{3} \left(y_1 + y_{2n+1} + 4 \sum_{i=1}^n y_{2i} + 2 \sum_{i=1}^n y_{2i-1} \right)$$

Gợi ý code MATLAB: Cho bảng dữ liệu

$$\frac{x|x_1 \quad x_2 \quad \cdots \quad x_{2n+1}}{y|y_1 \quad y_2 \quad \cdots \quad y_{2n+1}}.$$

Tính gần đúng tích phân $\int_a^b f(x)dx$ sử dụng công thức Simpson.

```
function [Iss, ss] = Simpson(x, y)
% số mốc chia là số lẻ
h = x(2)-x(1);
% bước lưới mịn
Iss = h/3*(y(1)+y(end)+2*sum(y(3:2:end-2))+4*sum(y(2:2:end-1)));
% bước lưới thưa
I2h = 2*h/3*(y(1)+y(end)+2*sum(y(5:4:end-4))+4*sum(y(3:4:end-2)));
% đánh giá sai số theo 2 bước
```

```
ss = 1/15*abs(Iss - I2h);  
end
```

Bài tập

Bài tập 1.

Viết thuật toán và chương trình MATLAB tính gần đúng tích phân

$$\int_a^b f(x)dx$$

với số đoạn chia được nhập vào. Kiểm tra chương trình với $n = 12$ và

$$I = \int_0^2 \sqrt{x^3 + 3} dx.$$

Giải số phương trình vi phân

Nội dung thực hành

1. Viết chương trình MATLAB giải gần đúng bài toán Côsi sử dụng công thức Euler.
2. Viết chương trình MATLAB giải gần đúng bài toán Côsi sử dụng công thức RK4.

Mọi đóng góp xin liên hệ: tam.doduc@hust.edu.vn

Tóm tắt lý thuyết

Bài toán Cauchy

Cho $I = [x_0, X]$ là một đoạn con đóng trong $\mathbb{R}$. Tìm hàm vector $y \in C[I, \mathbb{R}^k]$ sao cho

$$\begin{cases} \frac{dy}{dx} = f(x, y), x \in I \\ y(x_0) = y_0. \end{cases}$$

Chia đều đoạn I thành n đoạn có độ dài $h = (X - x_0)/n$ bởi các điểm $x_i, i = \overline{0, n}$. Kí hiệu $y_i \approx y(x_i), \quad \forall i = \overline{0, n}$.

Công thức Euler hiện

$$y_{i+1} = y_i + S_i = y_i + hf(x_i, y_i).$$

Công thức Euler ẩn

$$\begin{cases} t_{i,0} = y_i + hf(x_i, y_i) \\ t_{i,k+1} = y_i + hf(x_{i+1}, t_{i,k}) \\ y_{i+1} = y_i + hf(x_{i+1}, t_{i,k+1}) \end{cases}$$

Quy ước, nếu không nói rõ yêu cầu sai số, khi giải phương trình, chỉ cần lặp 1 lần, tức là dùng công thức trên bỏ qua công thức ở giữa.

Công thức hình thang

$$\begin{cases} t_{i,0} = y_i + hf(x_i, y_i) \\ t_{i,k+1} = y_i + \frac{h}{2} f(x_i, y_i) + \frac{h}{2} f(x_{i+1}, t_{i,k}) \\ y_{i+1} = y_i + \frac{h}{2} f(x_i, y_i) + \frac{h}{2} f(x_{i+1}, t_{i,k+1}) \end{cases}$$

Quy ước, nếu không nói rõ yêu cầu sai số, khi giải phương trình, chỉ cần lặp 1 lần, tức là dùng công thức trên bỏ qua công thức giữa.

Công thức RK3

$$\left\{ \begin{array}{l} k_{1,i} = y_i + hf(x_i, y_i) \\ k_{2,i} = y_i + hf\left(x_i + \frac{h}{2}, y_i + \frac{k_{1,i}}{2}\right) \\ k_{3,i} = y_i + hf(x_i + h, y_i - k_{1,i} + 2k_{2,i}) \\ y_{i+1} = y_i + \frac{1}{6}(k_{1,i} + 4k_{2,i} + k_{3,i}) \end{array} \right.$$

Công thức RK4

$$\left\{ \begin{array}{l} k_{1,i} = y_i + hf(x_i, y_i) \\ k_{2,i} = y_i + hf\left(x_i + \frac{h}{2}, y_i + \frac{k_{1,i}}{2}\right) \\ k_{3,i} = y_i + hf\left(x_i + \frac{h}{2}, y_i + \frac{k_{2,i}}{2}\right) \\ k_{4,i} = y_i + hf(x_i + h, y_i + k_{3,i}) \\ y_{i+1} = y_i + \frac{1}{6}(k_{1,i} + 2k_{2,i} + 2k_{3,i} + k_{4,i}) \end{array} \right.$$

Gợi ý code MATLAB phương pháp Euler

```
function [t, y] = euler(fun, t0, tn, y0, h)
%fun là kiểu handle function fun = @(t, y) biểu thức
t = t0:h:tn; % chu y input sao cho tn = t0+nh
n = length(t);
y = zeros(1, n);
y(1) = y0;
for i = 1:n-1
    y(i+1) = y(i) + h* fun(t(i), y(i));
end
```

Gợi ý code MATLAB phương pháp Euler cải tiến

```
function [t, y] = euler_caitien(fun, t0, tn, y0, h)
%fun là kiểu handle function fun = @(t, y) biểu thức
t = t0:h:tn; % chu y input sao cho tn = t0+nh
n = length(t);
y = zeros(1, n);
y(1) = y0;
for i = 1:n-1
    y(i+1) = y(i) + h/2* (fun(t(i), y(i))+...
        fun(t(i+1), y(i)+h*fun(t(i), y(i))));
end
end
```

Gợi ý code MATLAB phương pháp RK3

```
function [x, y] = rk3(fun, x0, X, y0, h)
% fun = @(x,y)expression
x = x0:h:X;
```

```

y = zeros(size(x));
y(1) = y0;
for i = 1:length(x)-1
    k1 = h*fun(x(i), y(i));
    k2 = h*fun(x(i)+h/2, y(i)+k1/2);
    k3 = h*fun(x(i+1), y(i)-k1+2*k2);
    y(i+1) = y(i) +1/6*(k1+4*k2+k3);
end
end

```

Bài tập

Bài tập 1.

Viết thuật toán và chương trình cho phương pháp RK4. Kiểm tra chương trình với bài toán sau:

$$\begin{cases} y' = xy^3 + 3x + 1.5x - 1 \\ y(1) = 0.25 \end{cases}.$$

Dùng công thức Runge-Kutta 4 để tính gần đúng giá trị $y(1.2)$ với bước $h = 0.2$.